

电子线路设计与测试 (二)

EDA软件使用与组合逻辑设计

电子线路设计与测试 (二)

EDA软件使用与组合逻辑设计

使用ISE软件设计、仿真、下载实现

- (1) 一位大小比较器 (三灯指示, 大于, 等于, 小于)**
- (2) 求反加1**
- (3) 3线-8线译码器**
- (4) 8线-3线优先编码器**

检查《电子线路设计、测试与实验 (二)》单元三、四、五测验结果

电子线路设计与测试 (二)

EDA软件使用与组合逻辑设计

选做：

使用ISE软件设计、仿真、下载实现

(1) 二进制-ASCII码转换表 (p147, EDA实验内容2)

(2) 数码管译码显示

(3) 文字显示器 (HELLO, HI, HAHA, HEHEA)

电子线路设计与测试 (二)

电子实验报告内容：

一、实验任务

二、设计平台

(1) 软件设计流程及作用

(2) 硬件平台简要介绍

三、设计源文件及注释

四、仿真文件及注释

五、仿真波形及分析

六、结果说明

七、反思及总结 (选)

实验目的

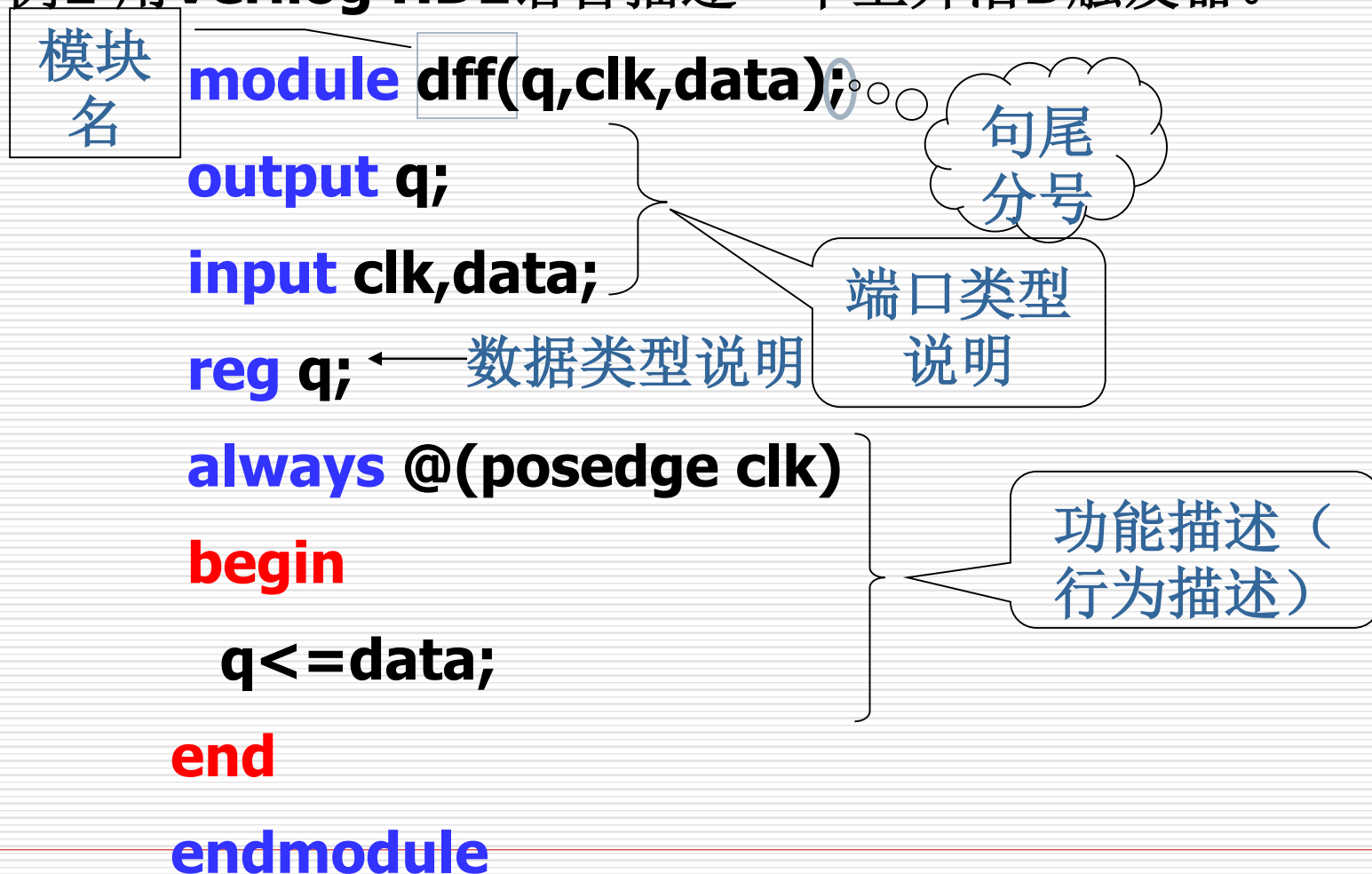
- **学习使用verilog HDL描述数字逻辑电路与系统的方法；**
- **了解并掌握采用可编程逻辑器件实现数字电路与系统的方法；**
- **学习并掌握采用Xilinx_ISE软件开发可编程器件的过程；**

自学内容：第7章，附录B

(一) Verilog HDL 硬件描述语言简介

Verilog HDL的基本结构

例1 用Verilog HDL语言描述一个上升沿D触发器。



(一) Verilog HDL 硬件描述语言简介

标识符：由字母或下划线开头

关键词：**Verilog HDL** 内部已使用的词。关键词都是小写。

变量的数据类型

wire [n-1: 0] 数据名1, 数据名2,, 数据名n;

或

wire [n: 1] 数据名1, 数据名2,, 数据名n;

reg [n-1: 0] 数据名1, 数据名2,, 数据名n;

或

reg [n: 1] 数据名1, 数据名2,, 数据名n;

在“**always**”模块内赋值的每一个信号都必须定义成**reg**型。

数据流描述方式

例: **module decoder(A,B,EN,Z);**
 input A,B,EN;
 output[3:0] Z;
 wire Abar,Bbar;
 assign Abar=~A;
 assign Bbar=~B;
 assign Z[0]=~(Abar&Bbar&EN);
 assign Z[1]=~(Abar&B&EN);
 assign Z[2]=~(A&Bbar&EN);
 assign Z[3]=~(A&B&EN);
 endmodule

连线类型说明与assign连续赋值语句

连线类型 **[位宽说明]** 连线名表项;

assign assign-statement;

例: `wire c;`

`assign c=a&b;`

行为描述方式

使用下述语句描述

1) **initial** 语句

2) **always** 语句

在“**always**”模块内赋值的每一个信号都必须定义成**reg**型。

Verilog HDL行为描述

过程块的组成:

过程语句 @ (事件控制敏感表)

块语句开始标识符: 块名

块内局部变量说明

一条或多条过程赋值或高级程序语句

块语句结束标识符

过程语句 **initial** 与 **always**

(1) **initial**后面的块语句只执行一次，而**always**则重复执行。

(2) **initial**不带触发条件，而**always**通常带有触发条件。

(3) 可以有多个**initial**语句和**always**语句，它们之间相互独立，并行运行。

事件控制敏感表

- 1) @ (信号名) //当信号名所确定的信号发生变化时
 - 2) @ (**posedge** 信号名) //当信号名所确定的信号上升沿到来时
 - 3) @ (**negedge** 信号名) //当信号名所确定的信号下降沿到来时
 - 4) @ (敏感事件**1 or 敏感事件2 or 敏感事件3.....**)
-

块语句

串行块 **begin-end**

块中语句按串行方式顺序执行

并行块 **fork-join**

块中语句按并行方式同时执行

赋值语句

基本形式： <寄存器变量>=<表达式>;

阻塞型过程赋值与非阻塞型过程赋值

赋值算符“=”：阻塞型过程赋值算符

前一条语句没有完成赋值过程之前，后面的语句不可能被执行。

赋值算符“<=”：非阻塞型过程赋值算符

一条非阻塞型赋值语句的执行，并不会影响块中其它语句的执行

高级程序语句

(1) **if-else** 条件语句

if (条件表达式) 块语句**1**

else if (条件表达式**2**) 块语句**2**

.....

else if (条件表达式**n**) 块语句**n**

else 块语句**n+1**

高级程序语句

例：具有同步清零功能的上升沿D触发器

```
module dff(q,d,clr,clk);  
output q;  
input d,clr,clk;  
reg q;  
always @(posedge clk)  
begin  
    if (!clr) q<=0;  
    else q<=d;  
end  
endmodule
```

高级程序语句

例：具有异步清零功能的上升沿**D**触发器

```
module dff(q,d,clr,clk);  
output q;  
input d,clr,clk;  
reg q;  
always @( negedge clr or posedge clk)  
begin  
    if (!clr) q<=0;  
    else q<=d;  
end  
endmodule
```

高级程序语句

(2) **case** 语句

case (敏感表达式)

值**1**: 块语句**1**

值**2**: 块语句**2**

.....

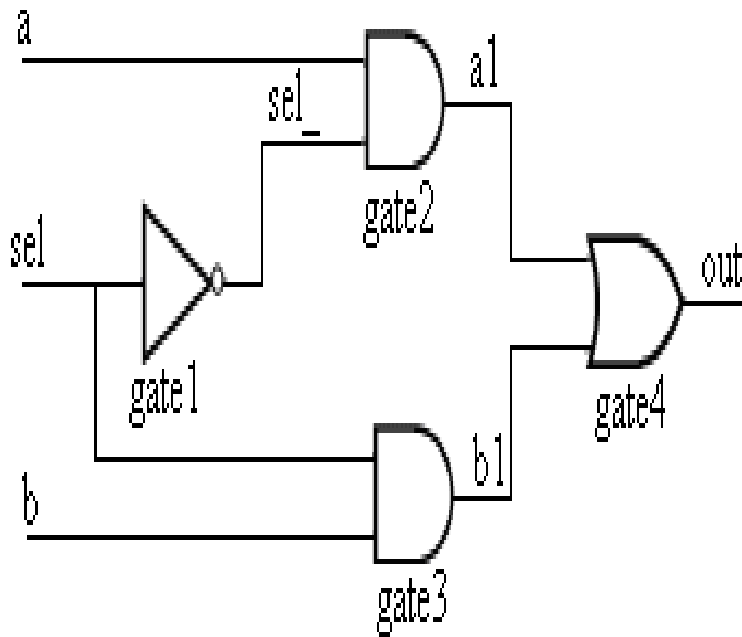
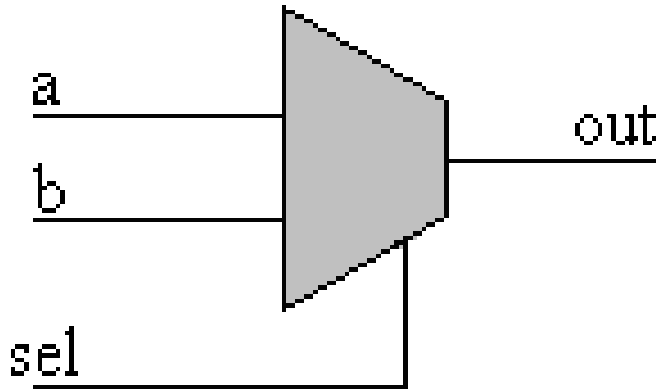
值**n**: 块语句**n**

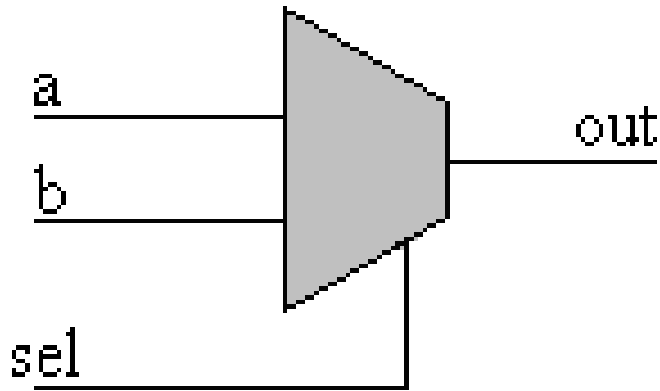
default:块语句**n+1**

endcase

方法一：

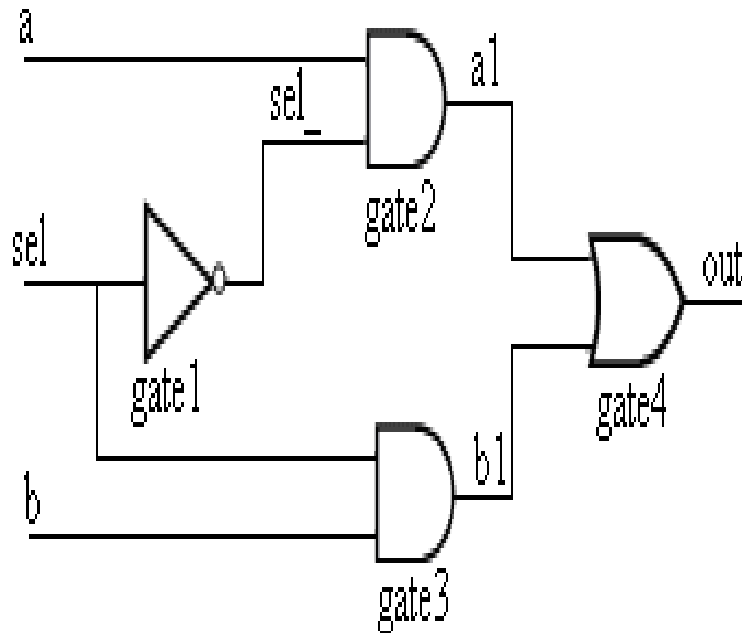
```
module mux2_1(out, a, b, sel);  
    output out;  
    input a, b;  
    input sel;  
  
    reg out;  
    always @(sel or a or b)  
    begin  
        if (sel)  
            out = b;  
        else    out = a;  
    end  
endmodule
```





方法二:

```
module mux2_1(out, a, b, sel);
    output out;
    input a, b;
    input sel;
    reg out;
    always @(sel or a or b)
    begin
        case (sel)
            1'b0 : out = a;
            1'b1 : out = b;
        endcase
    end
endmodule
```



Verilog HDL结构描述

基本门的调用

门名 调用名（端口名表项）

（**1**）门名：**14**种基本门级元件

（**2**）调用名与端口名表项：若同一个基本门在当前模块中被调用多次，可在一条调用语句中加以说明，中间以逗号相隔。

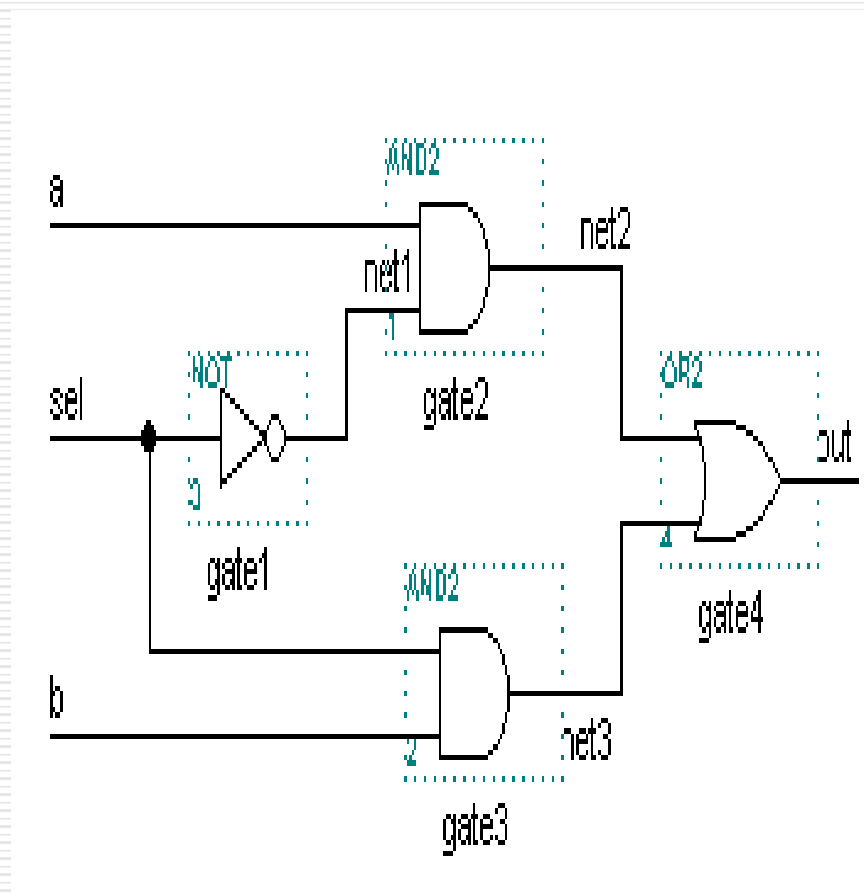
Verilog 基本门级元件

- 多输入门: **and**、**nand**、**or**、**nor**、**xor**、**xnor**
只有单个输出,**1**个或多个输入
 - 多输出门: **not**、**buf**
允许有多个输出,但只有一个输入
 - 三态门: **bufif0**、**bufif1**、**notif0**、**notif1**
有一个输出,一个数据输入和一个控制输入
-

结构化描述方式

基本门的调用

```
module
    mux(out,a,b,sel);
output out;
input a,b,sel;
not gate1(net1,sel);
and gate2(net2,a,net1);
and gate3(net3,b,sel);
or gate4(out,net2,net3);
endmodule
```



结构化描述方式

模块的调用

基本方式： 模块名 调用名（端口名表项）

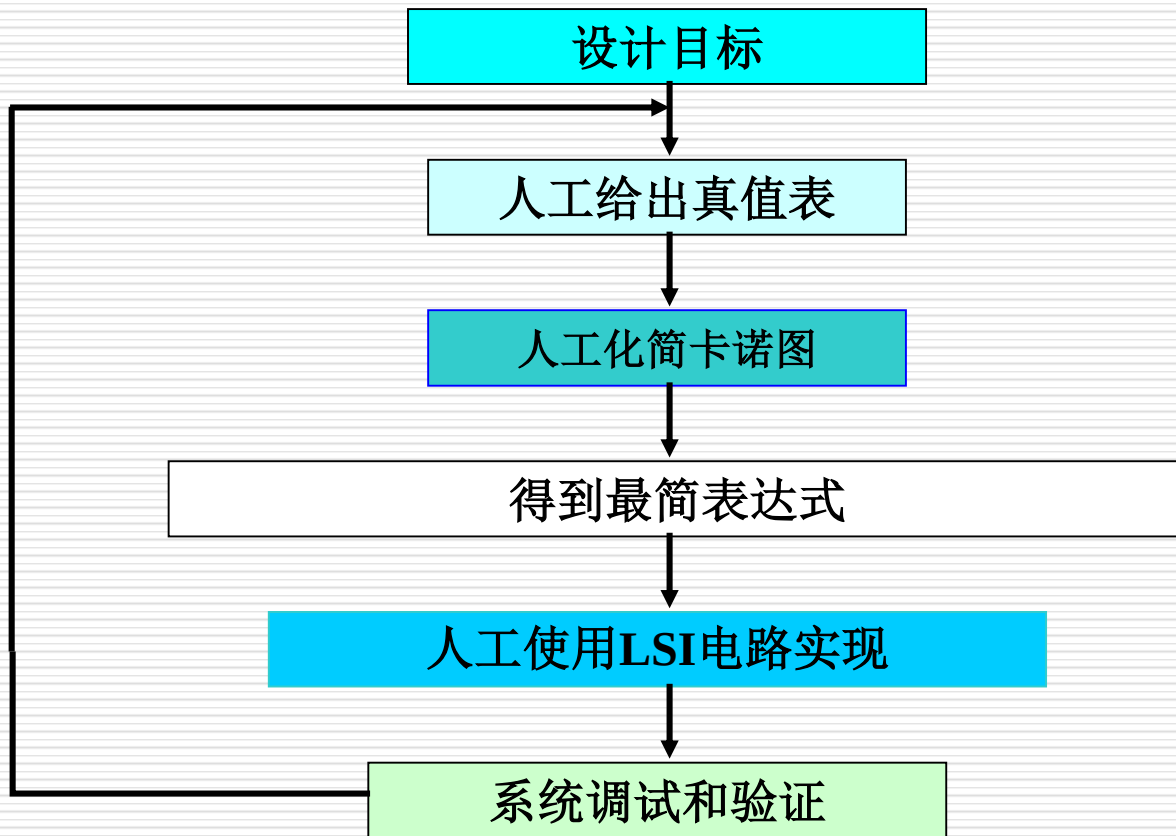
调用方式一： 位置对应调用方式

调用方式二： 端口名对应调用方式

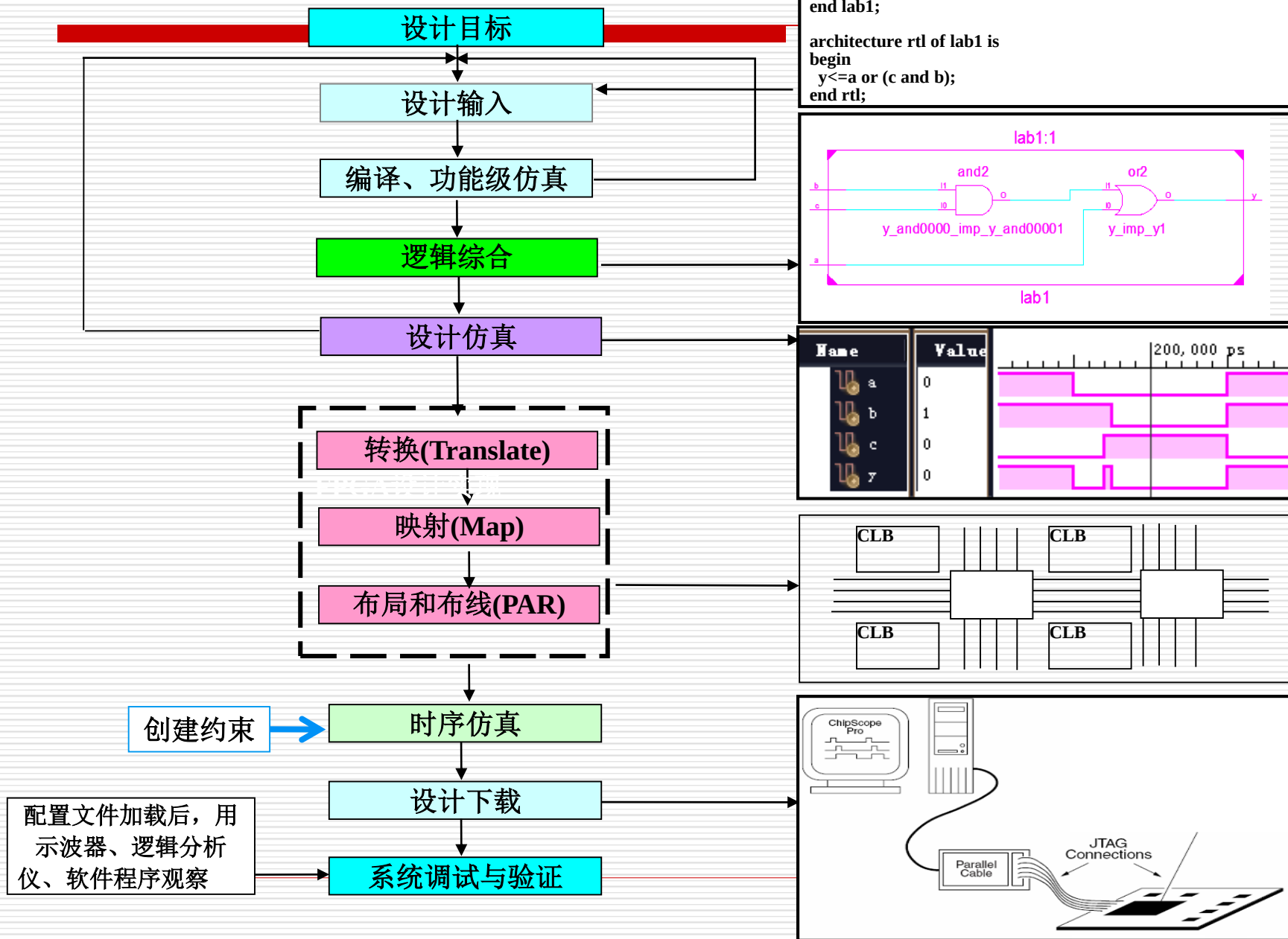
调用方式三： 存在不连接端口的调用方式

(二) 采用FPGA实现数字系统方法

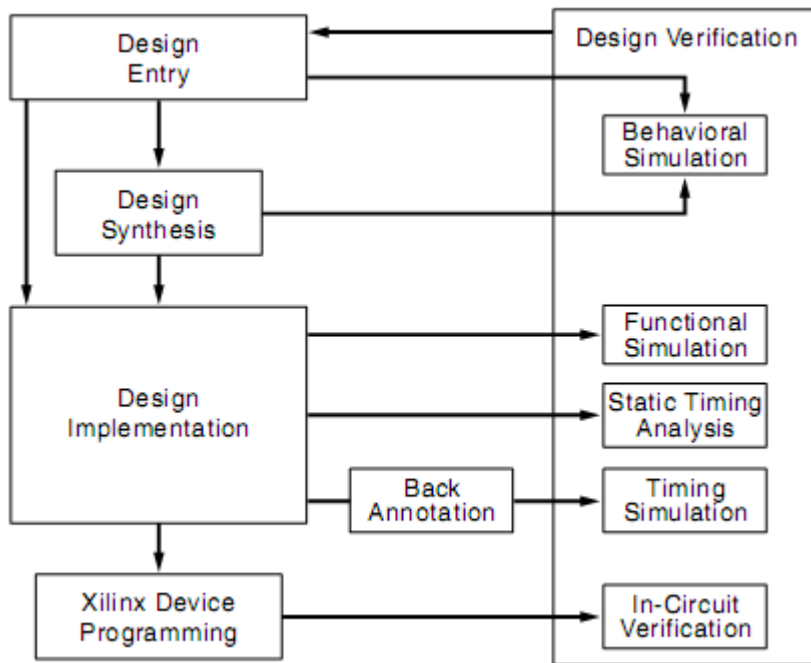
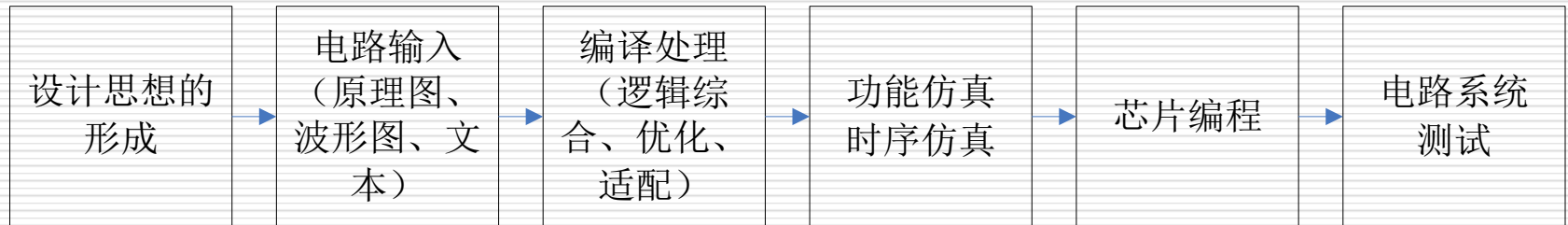
传统数字系统设计流程



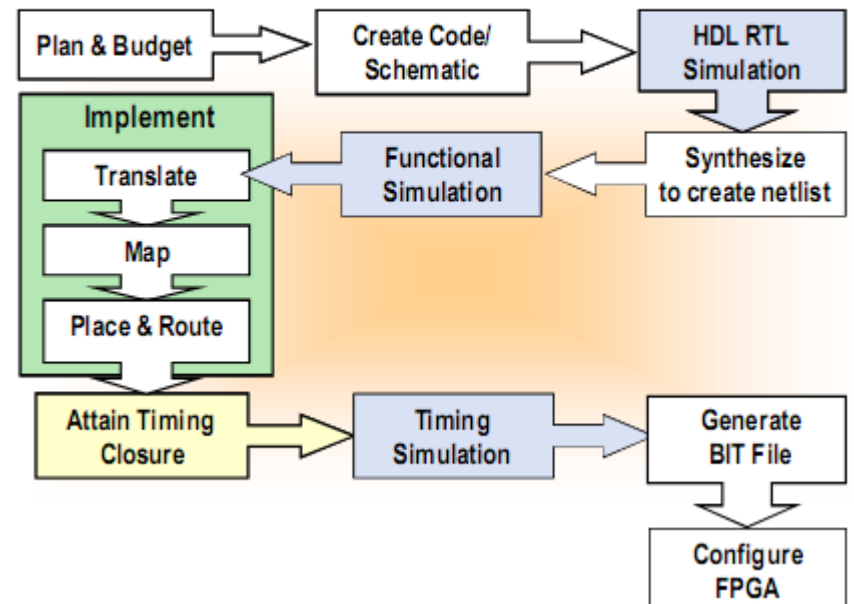
现代数字系统设计流程



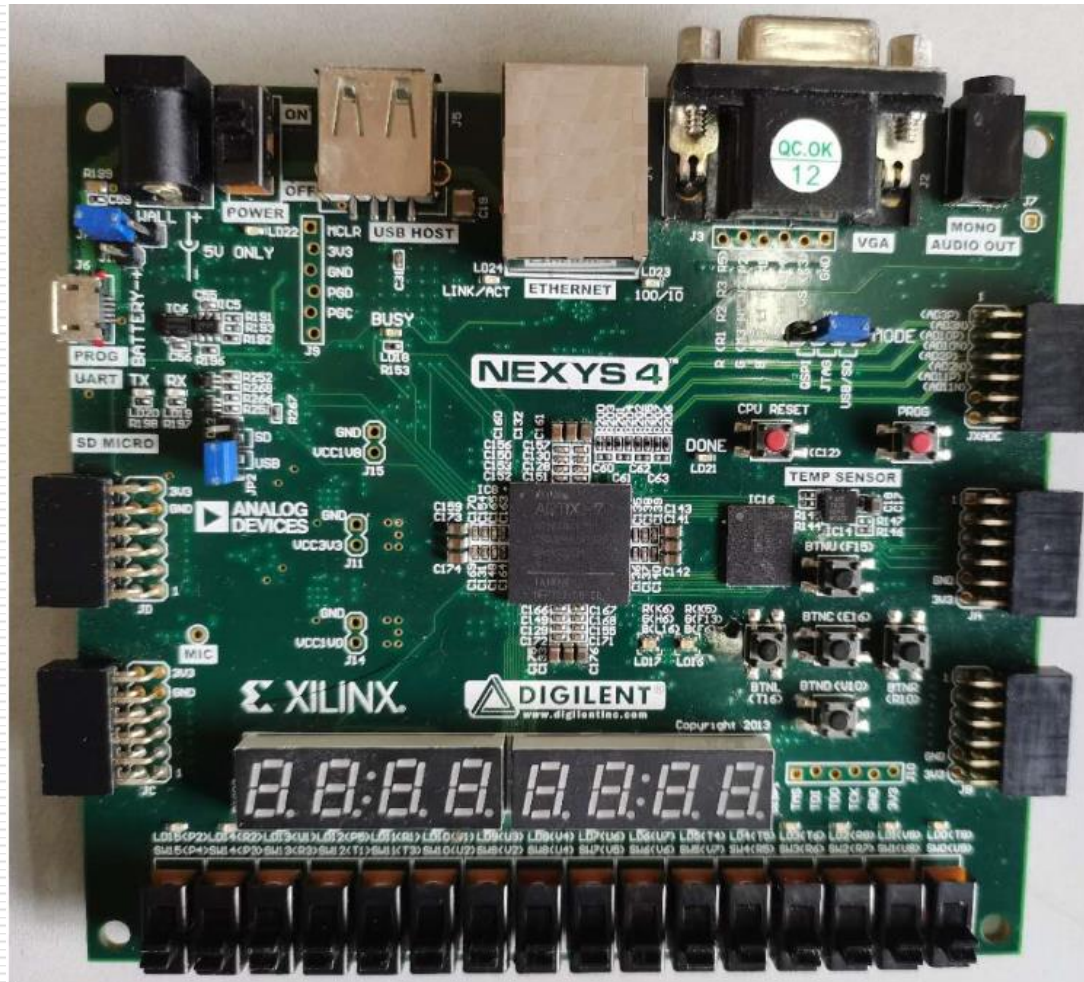
XilinxFPGA器件的开发流程



Xilinx Design Flow

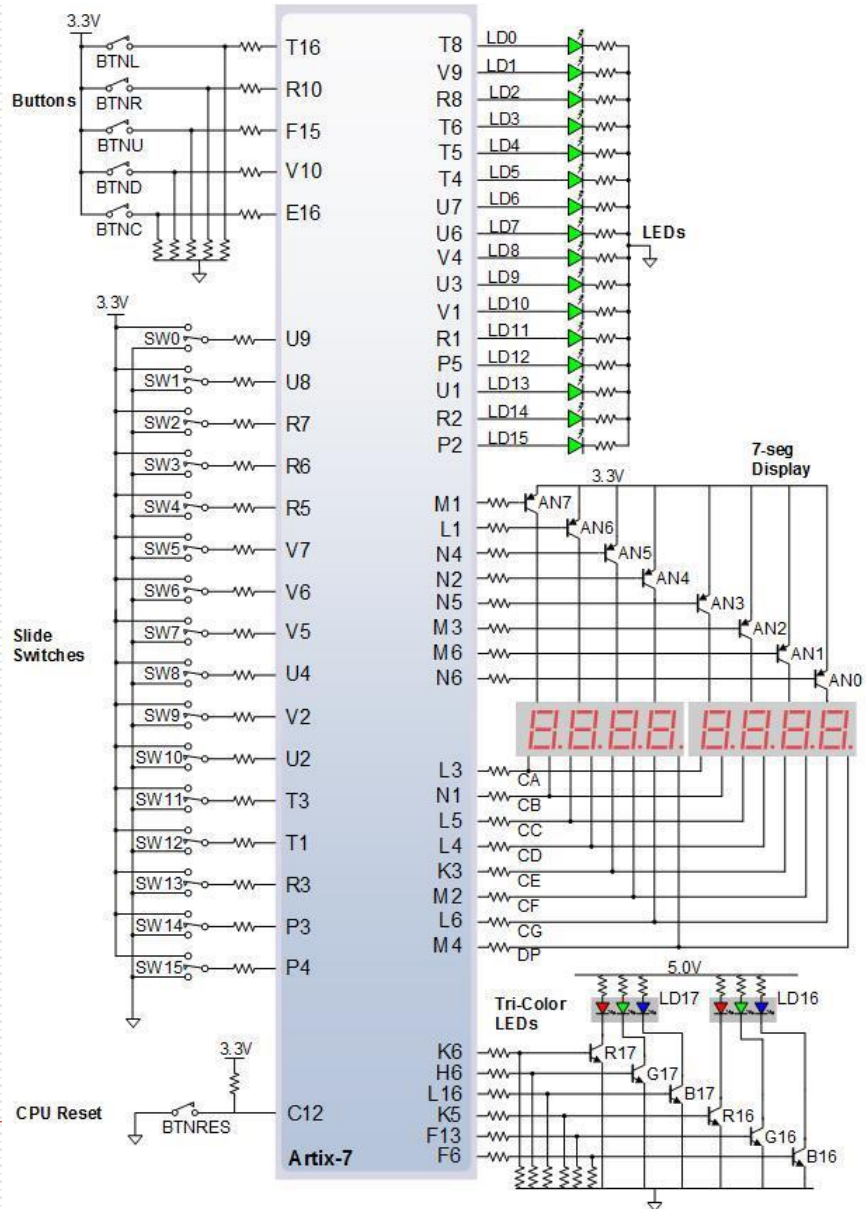


Xilinx开发板Nexys4



Xilinx part number :XC7A100T-1CSG324C

Nexys4实验板可用资源



(三) 采用Xilinx_ISE软件开发可编程器件

ISE开发环境

开启ISE14.7软件：开始→程序→Xilinx ISE Design Suite 14.7→ISE Design Tools→32-bit Project Navigator, 会出现ISE14.7 的画面.



ISE 设计流程：设计输入、综合、仿真、实现和下载

1.设计输入

文本编辑器（**Text Editor**）：**HDL**代码输入和查看报告

ECS（**Engineering Capture System**）：原理图编辑

Core Generator：生成**IP Core**

StateCAD：状态机设计

Constraint Editor：约束文件编辑

2.综合

包含**Xilinx**综合工具**XST**，内嵌**LeonardoSpectrum**和**Synplify**。

3.仿真

自带**ISESimulation**仿真工具，提供使用**Modelsim**仿真的接口。

4.实现

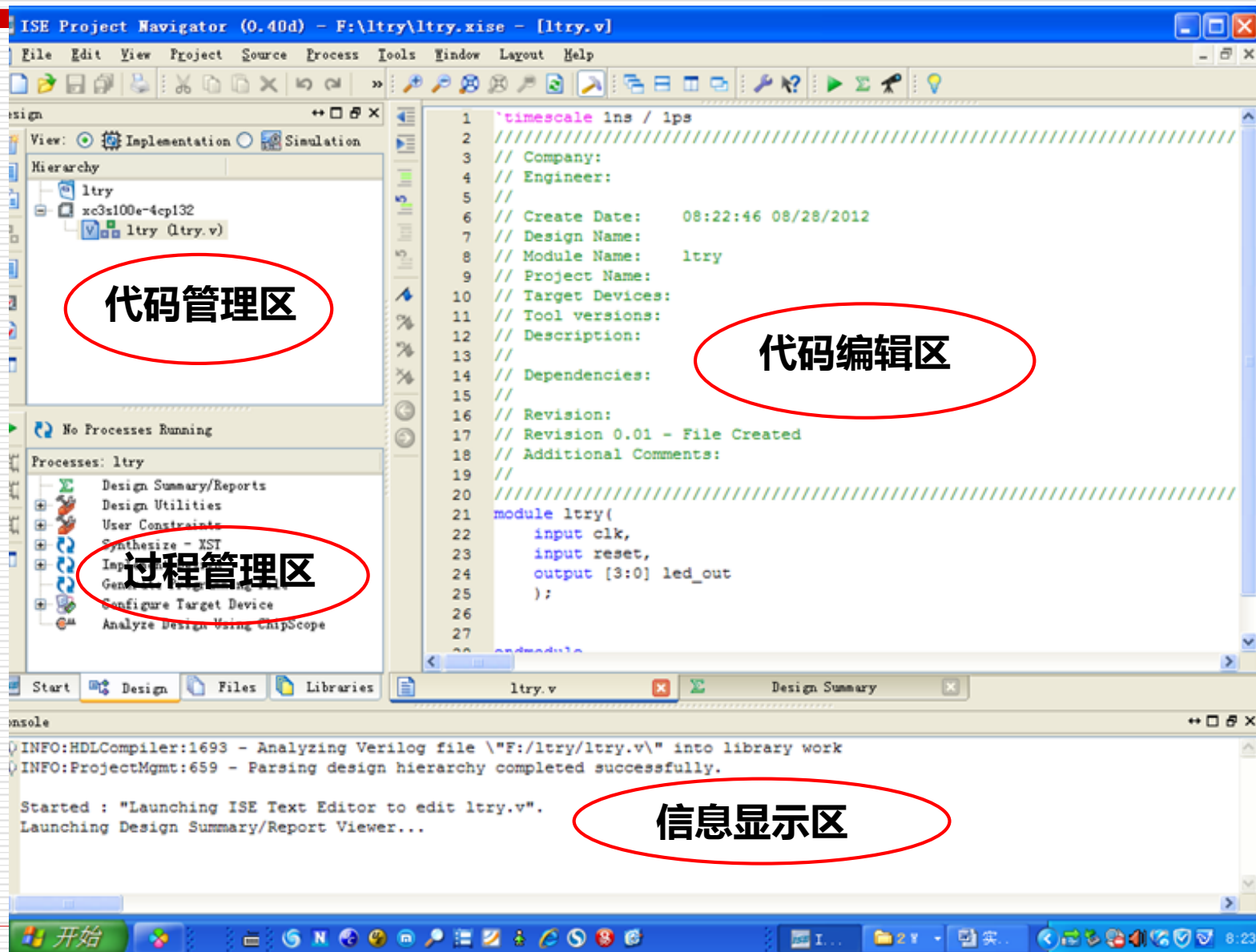
包括翻译、映射、布局布线等，还具备时序分析、引脚指定等高级功能。

5.下载

BitGen:将布局布线后的设计文件转换为位流文件

IMPACT：控制将程序烧写到**FPGA**芯片中去。

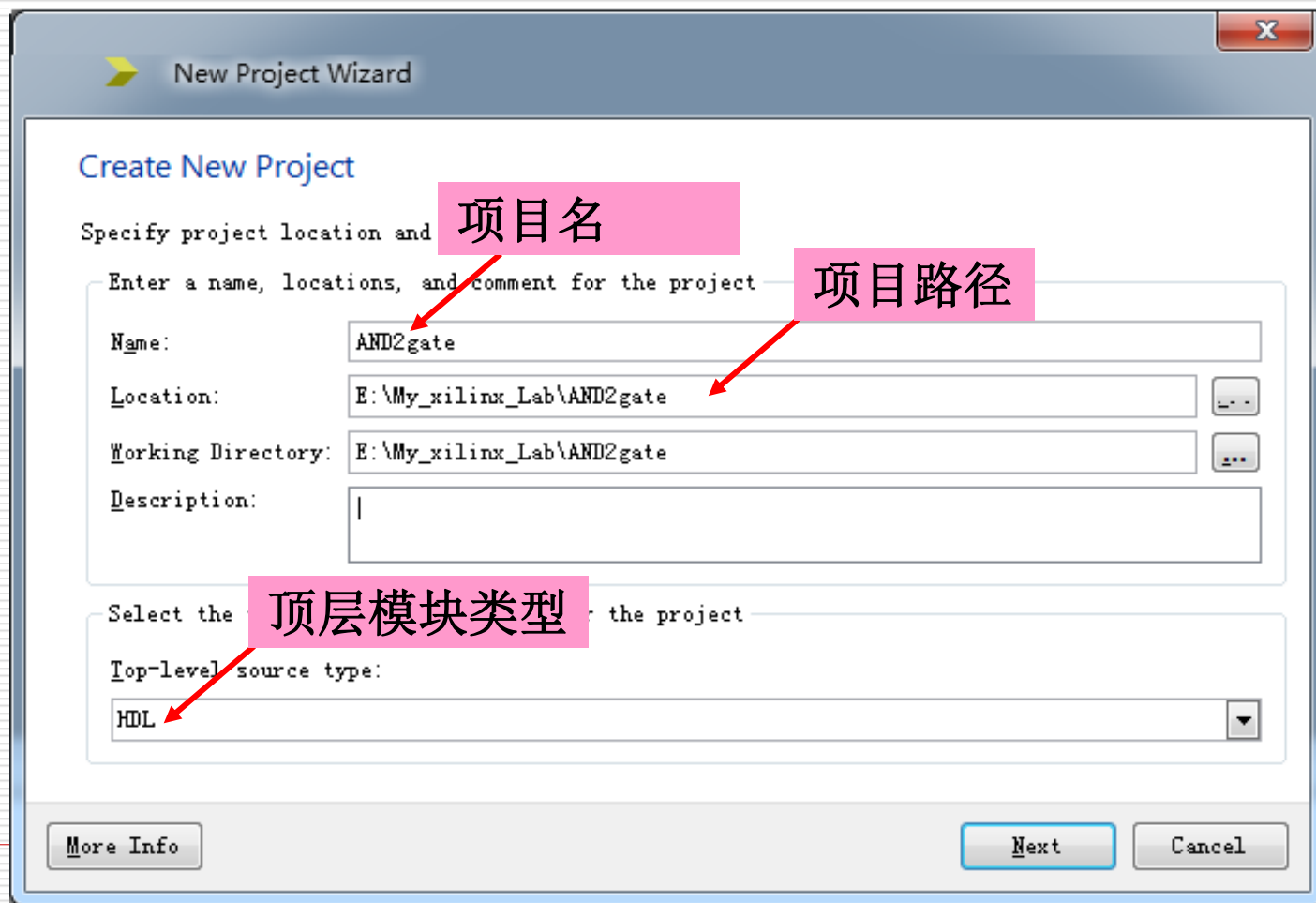
ISE开发环境



设计举例---两输入与门

1. 新建工程

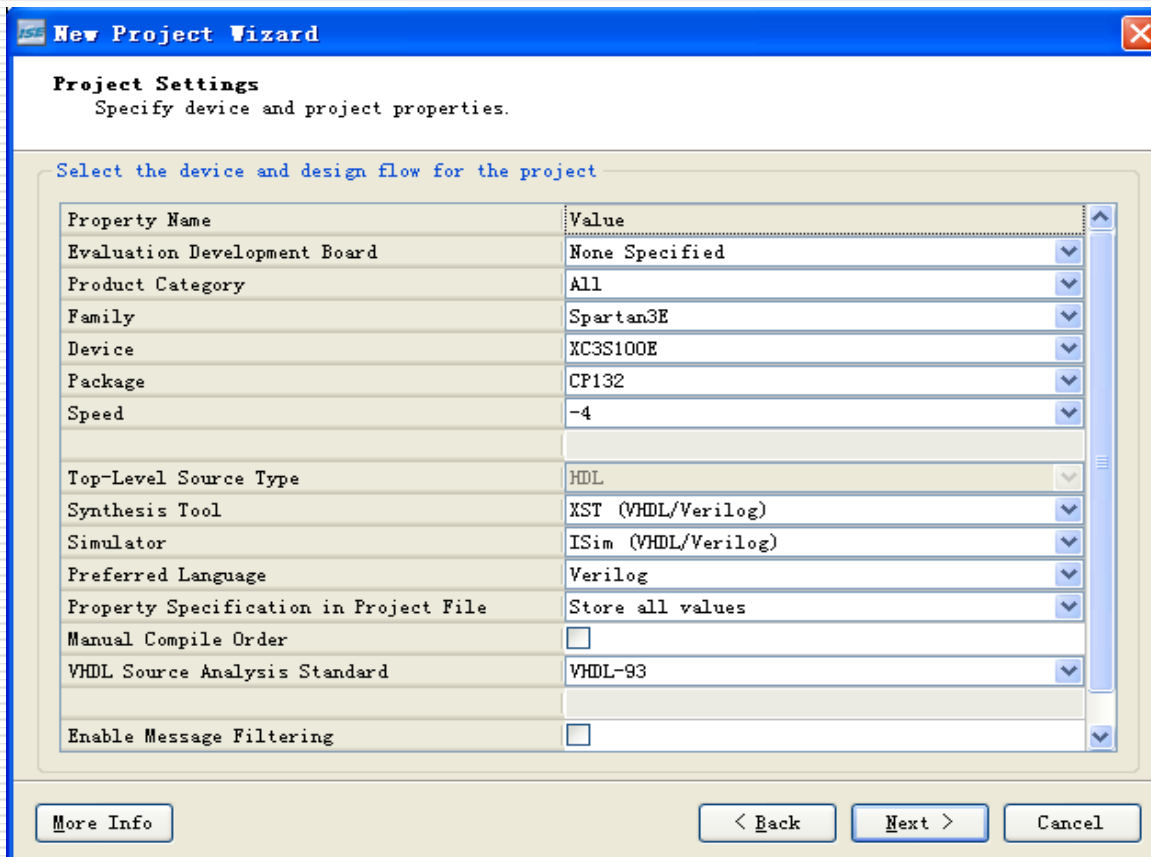
(1) 点击新的工程: **File -> New Project** 新建工程



流水灯---新建工程

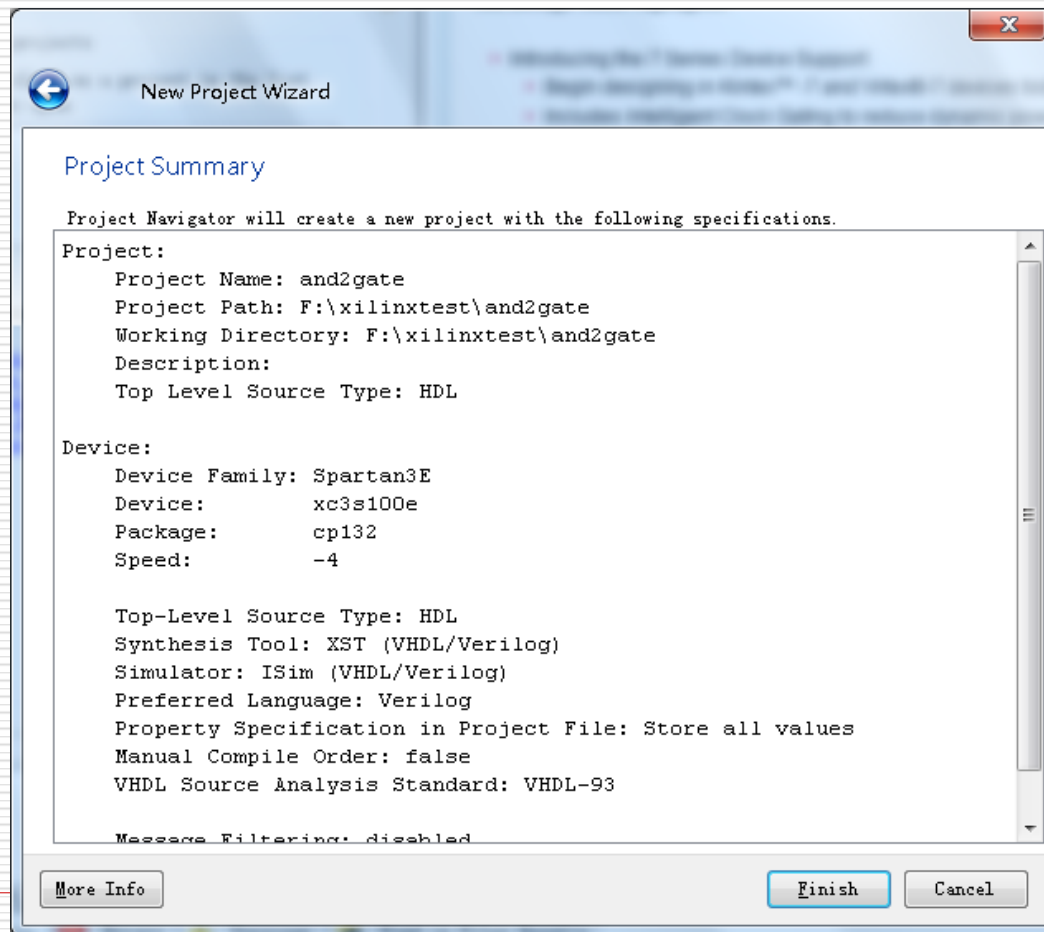
(2) 单击next,选择设计所用器件和工具。

Family（器件系列）：
Spartan3E;
Device（器件型号）：
XC3S100E
Package（器件封装）：
CP132
Speed grade（速度等级）：
-4
Synthesis Tool(综合工具):**XST**
(VHDL/Verilog)
Simulator(仿真器)：
Isim(VHDL/Verilog)



两输入与门---新建工程

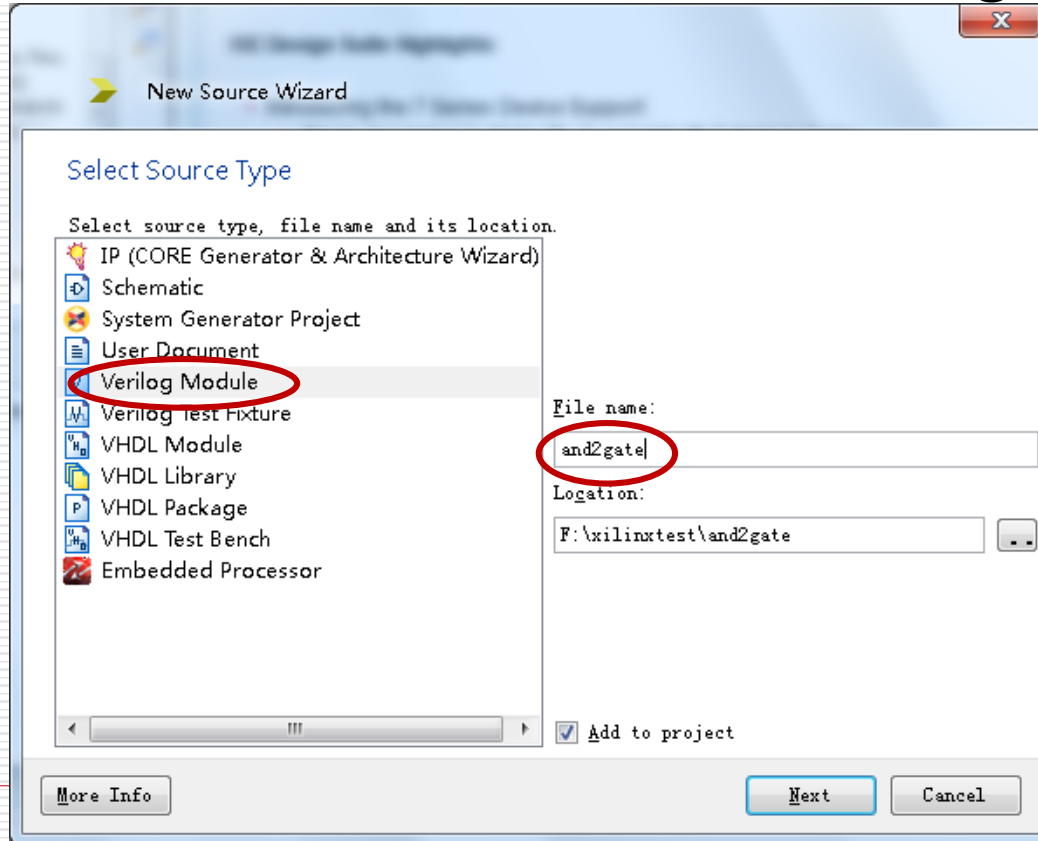
(3) 点击next. 此时出现此项目所有设定的信息, 若需重新设定,则可back. 若无误,则按finish



两输入与门---创建新的Verilog源文件

(1) 此时出现一个项目的架构,可以允许使用者开始进行项目的设计.

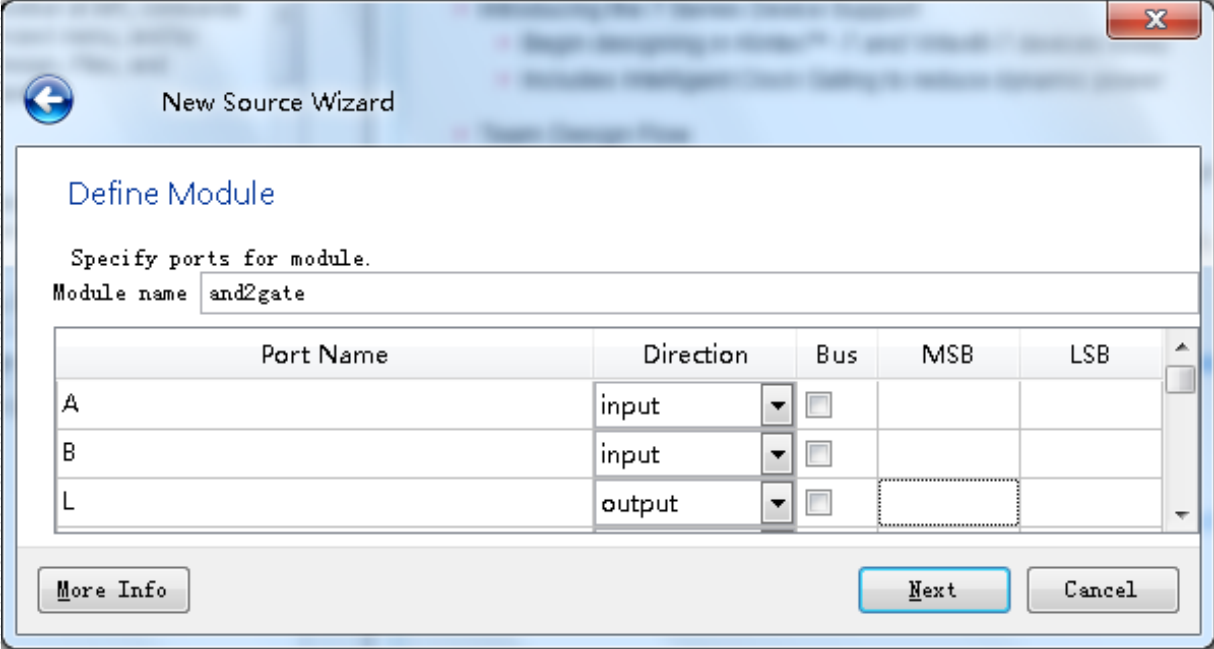
(2) 创建新的设计文件: Project -> New Source,选择 Verilog Module,并设定文件名称为and2gate



两输入与门---创建新的Verilog源文件

(3) 点击next,出现New Source Wizard,设定此设计的输入输出信号。

Port Name表示端口名称, Direction表示端口方向(可以选择input、output或者inout), MSB表示信号最高位, LSB表示信号最低位。对于单个信号, MSB和LSB不用填写



New Source Wizard

Define Module

Specify ports for module.

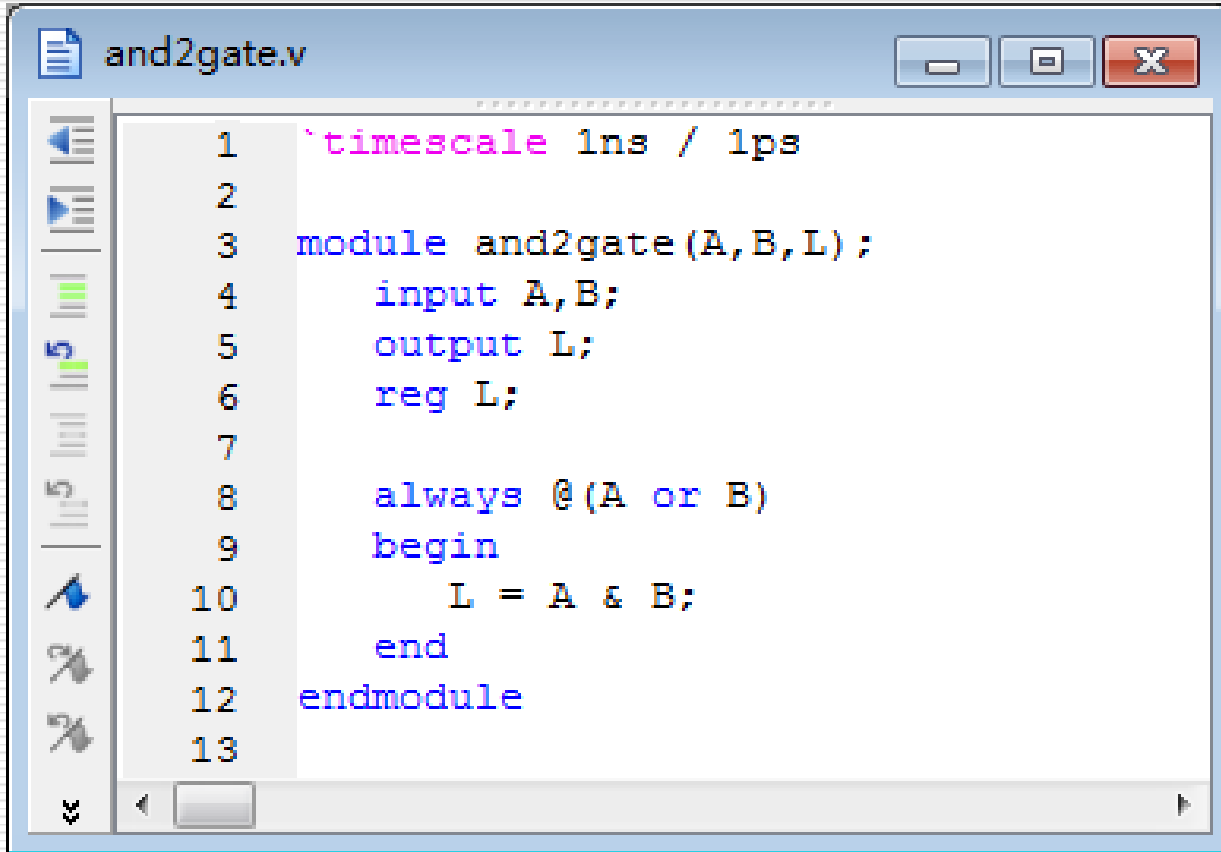
Module name

Port Name	Direction	Bus	MSB	LSB
A	input	☐		
B	input	☐		
L	output	☐		

两输入与门---创建新的Verilog源文件

(4) 点击next, 再点finish, 此时在Project Navigator右边的工作区可以看到and2gate.v的文件内容, 修改完善设计内容, 点击File -> Save保存文件。

`timescale编译指令指定仿真器的时间单位为1 ns。



```
1  `timescale 1ns / 1ps
2
3  module and2gate(A,B,L);
4      input A,B;
5      output L;
6      reg L;
7
8      always @(A or B)
9      begin
10         L = A & B;
11     end
12 endmodule
13
```

`timescale

<time1> /

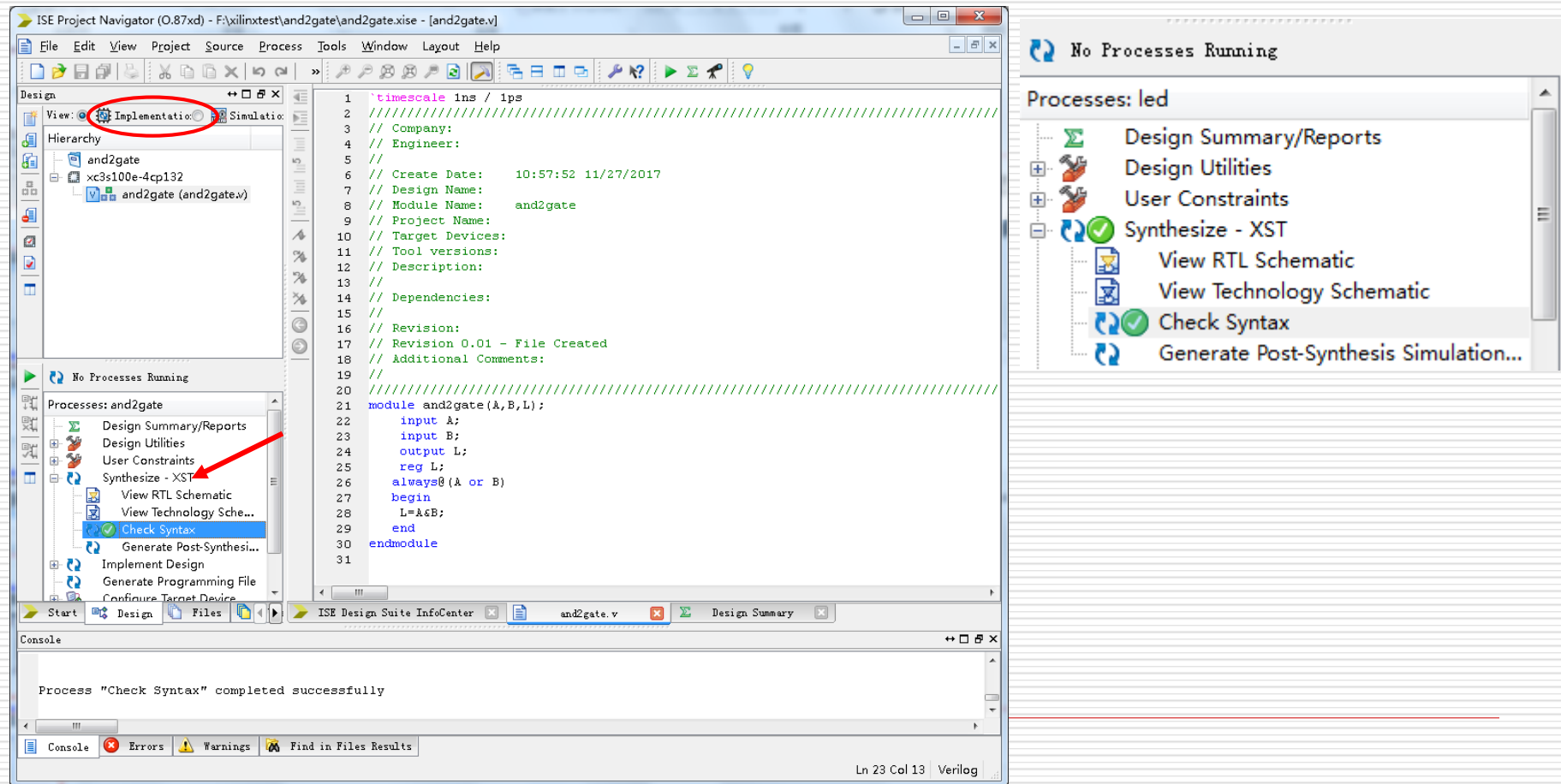
<time2>的含义

为: <time1>指明时间单位, 与测试文件中的“#延迟值”相关联

; <time2>指明仿真器使用的最小时间步长。

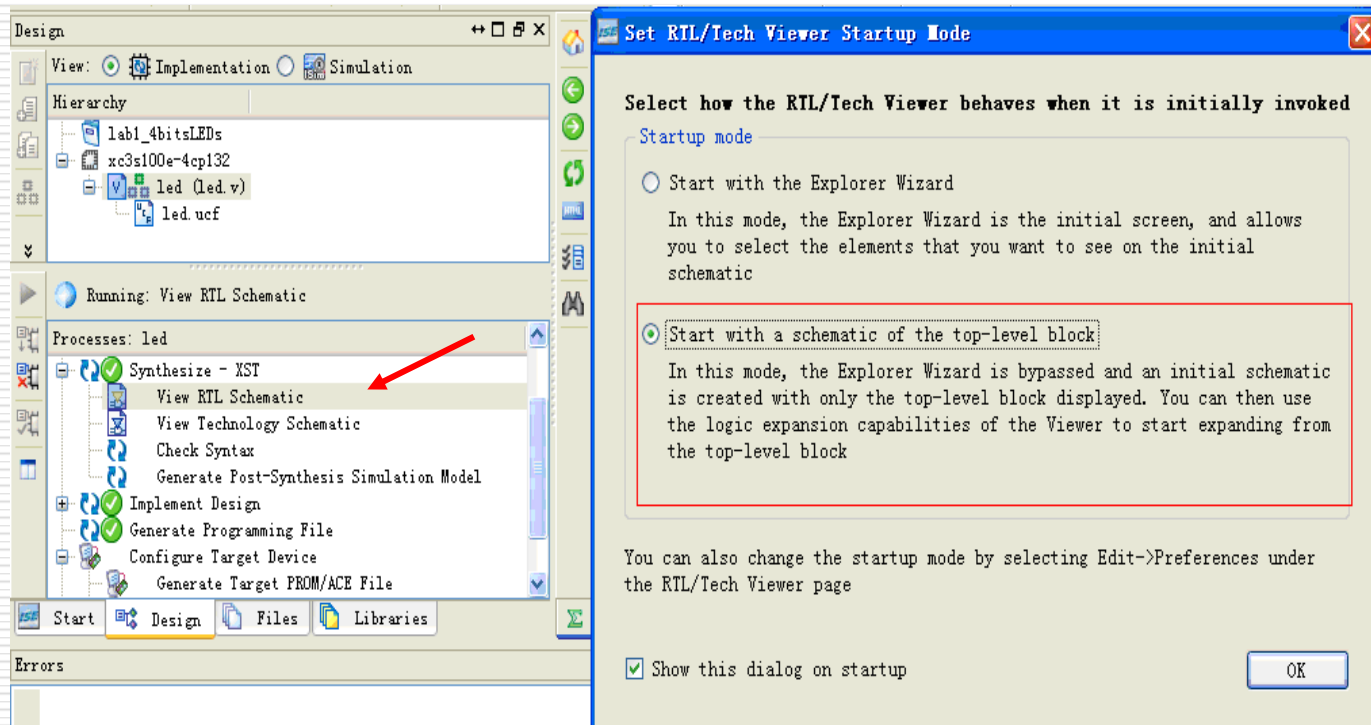
两输入与门---基于Xilinx XST的综合

(1) 代码写完之后，可进行句法检查。点开Synthesize-xst，双击Check Syntax。如果语法没错，是绿色的打勾，否则会有红色的错误提示。



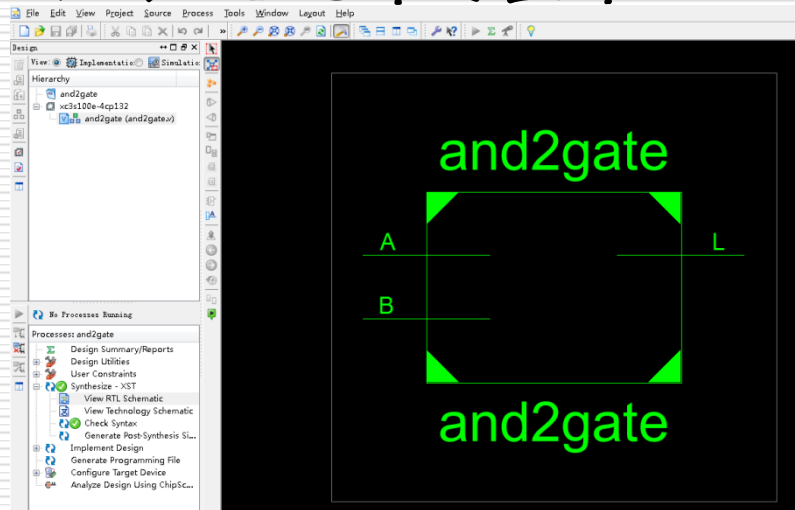
两输入与门---基于Xilinx XST的综合

(2) 双击Synthesize - xst进行综合（生成EDF文件），确认设计的正确与否，还可以查看设计RTL电路。点开综合选项，双击View RTL Schematic，并选择Start with a schematic of the top-level block选项

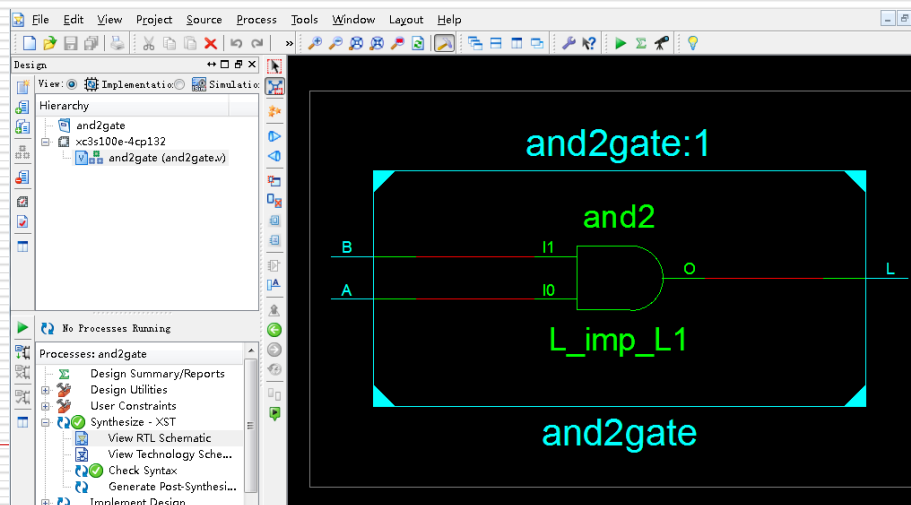


两输入与门---基于Xilinx XST的综合

(3) 点击OK，出现设计的整个电路模块图

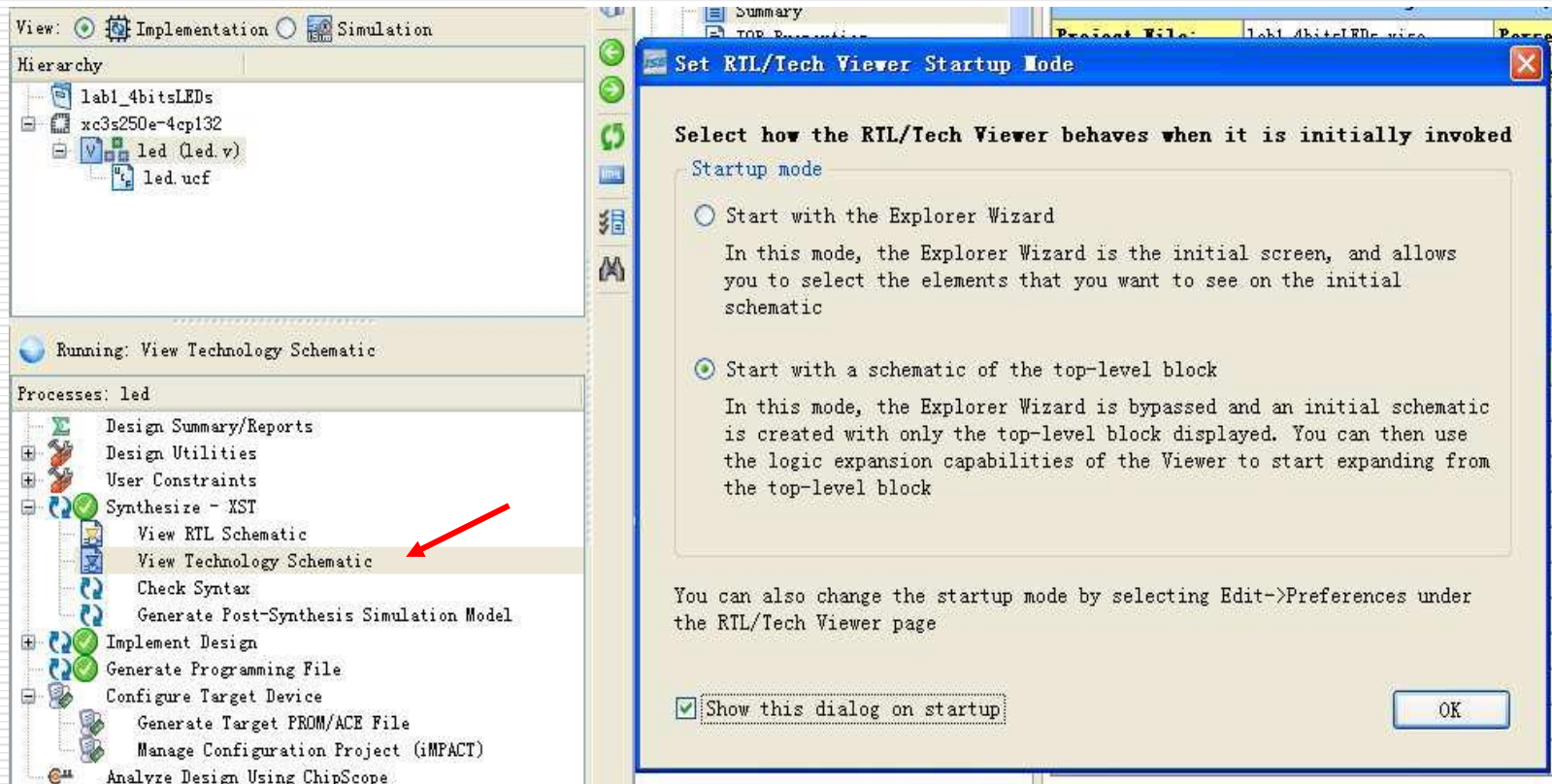


(4) 直接双击电路顶层，可查看内部电路模块



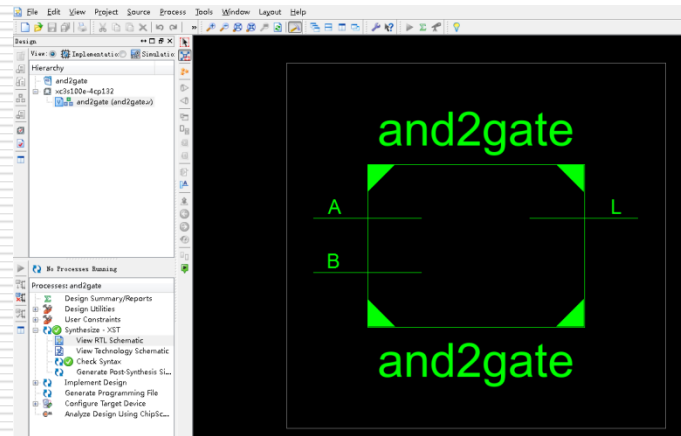
两输入与门——基于Xilinx XST的综合

(5) 如果需要查看设计内部具体有哪些实际资源组成，则可以点开综合选项，双击“view technology schematic”

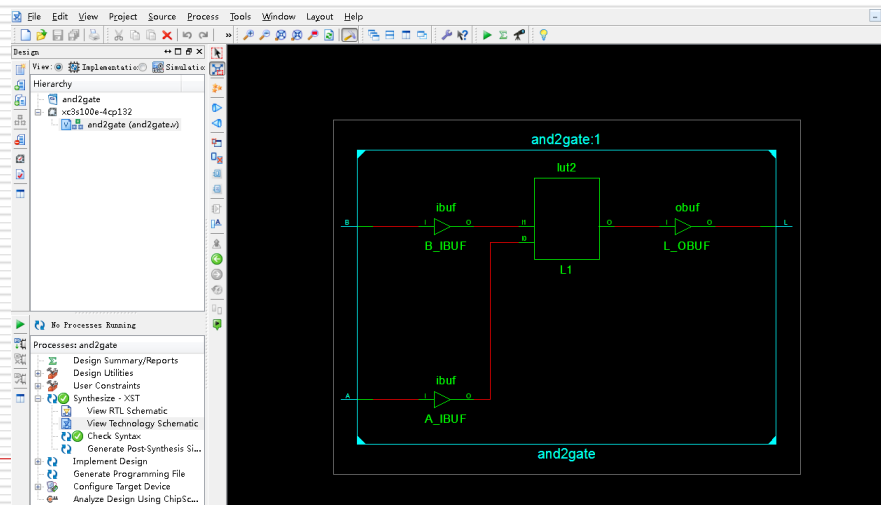


两输入与门---基于Xilinx XST的综合

(6) 点击OK，出现设计顶层



(7) 直接双击顶层电路，则可以看到设计的内部电路是由哪些资源组成的。

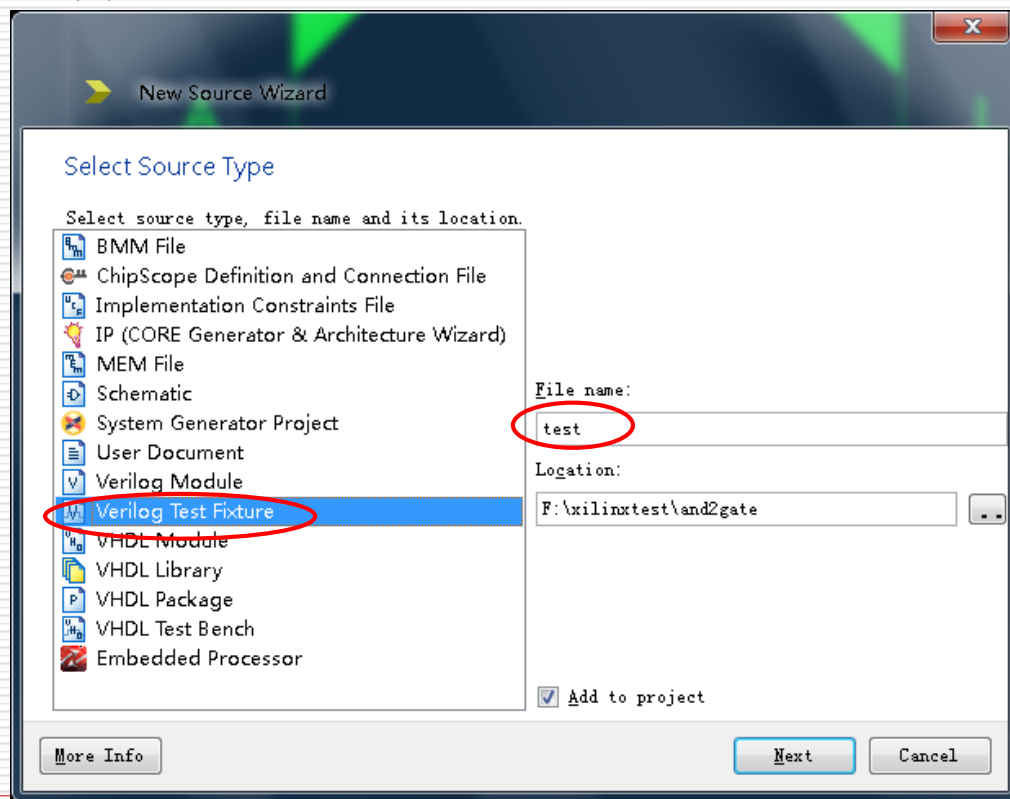
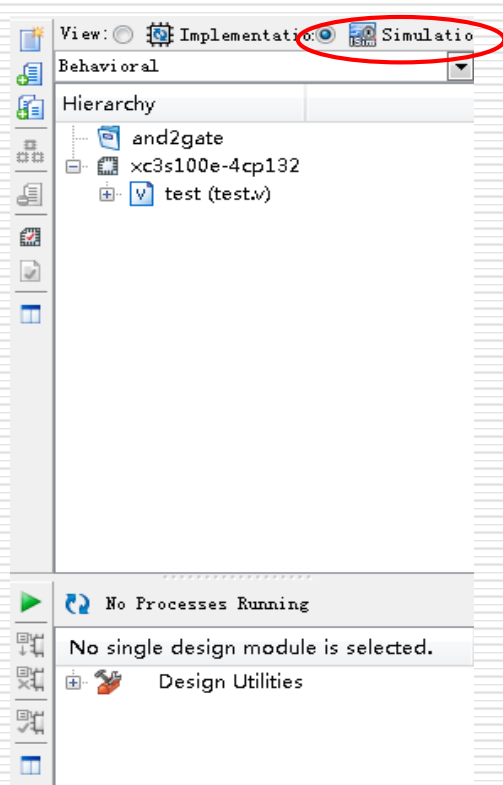


两输入与门---设计仿真

当检查输入源程序句法没有错误后，一般在设计综合之前做行为仿真，以验证代码的行为功能是否正确，利用测试模板(Testbench)来进行。

两输入与门——设计仿真

(1) 在源代码管理区，将“View”设置为“Simulation”
在任意位置单击鼠标右键，在弹出菜单中选择“New Source”，在源文件向导里，点击Verilog Test Fixture作为源文件类型，输入文件名称为test.



两输入与门——设计仿真

(2) 一直点击next,直到点击finish,自动生成test.v的测试模板。将测试文件补充完整:

```
`timescale 1ns / 1ps
module test;

    // Inputs
    reg A;
    reg B;

    // Outputs
    wire L;

    // Instantiate the Unit Under Test (UUT)
    and2gate uut (
        .A(A),
        .B(B),
        .L(L)
    );

    initial begin
        // Initialize Inputs
        A = 0;
        B = 0;

        // Wait 10 ns for global reset to finish
        #10;

        // Add stimulus here
        A = 0;
        B = 1;

        // Wait 10 ns for global reset to finish
        #10;
        A = 1;
        B = 0;

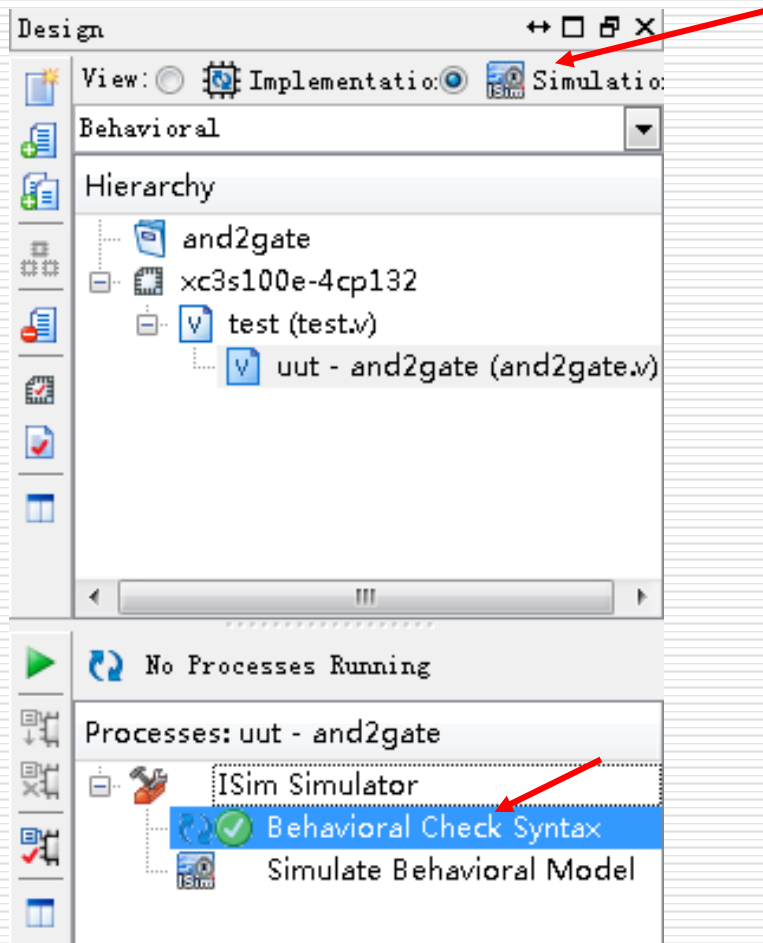
        // Wait 10 ns for global reset to finish
        #10;
        A = 1;
        B = 1;

        // Wait 10 ns for global reset to finish
        #10;

    end
endmodule
```

两输入与门——设计仿真

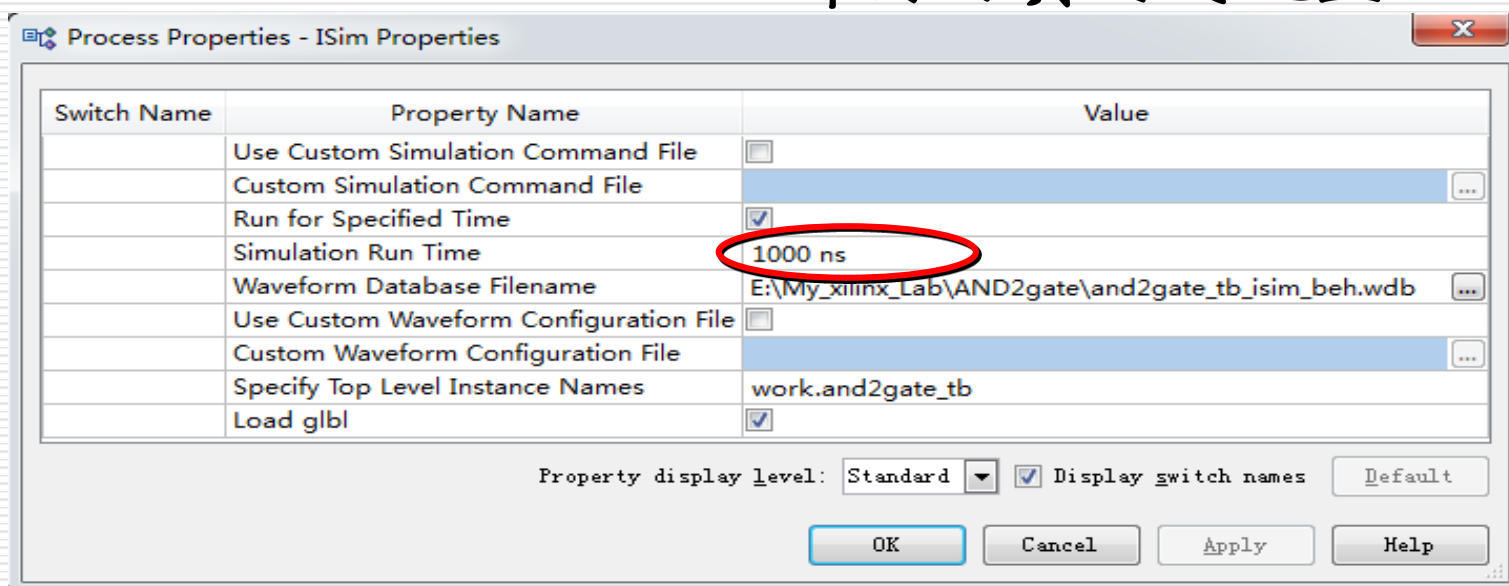
(3) 保存test.v。双击**Isim Simulator** -> Behavioral Check Syntax



两输入与门——设计仿真

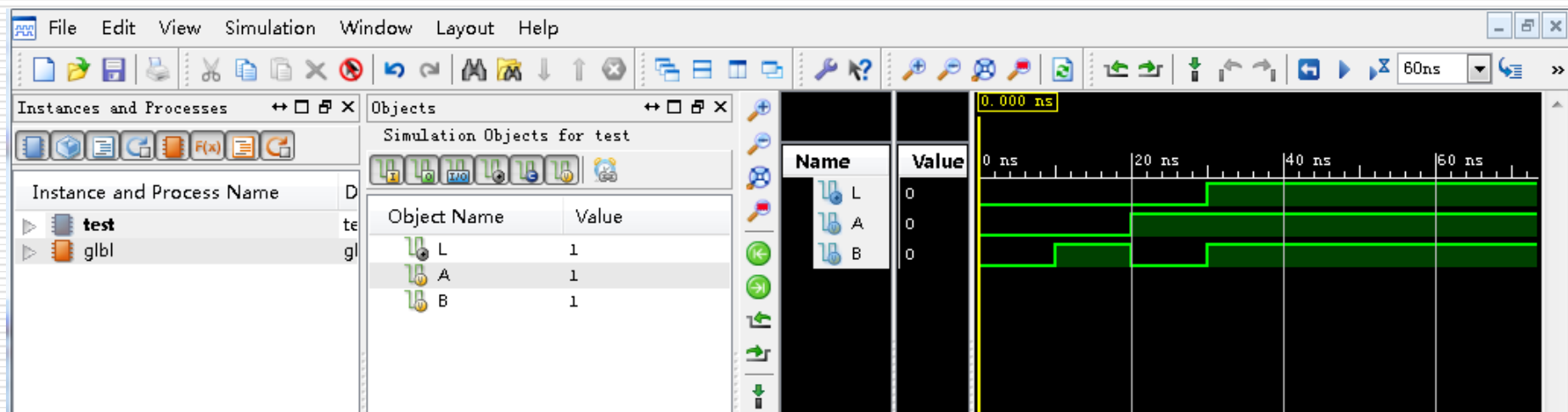
(4) 为控制Isim仿真器运行时间，选中**Isim Simulator** -> **Simulate Behavioral Model**，右键，选出弹出菜单的**Properties**项，在弹出的属性设置对话框的最后一行：

“**Simulation Run Time**”即为仿真时间设置。



两输入与门——设计仿真

(5) 双击 **Isim Simulator** -> Simulate Behavioral Model, ISE 仿真器打开并开始仿真, 将图形界面 **缩小** 到合适的界面, 其仿真结果如图所示:



(6) 关闭Isim仿真器并保存。

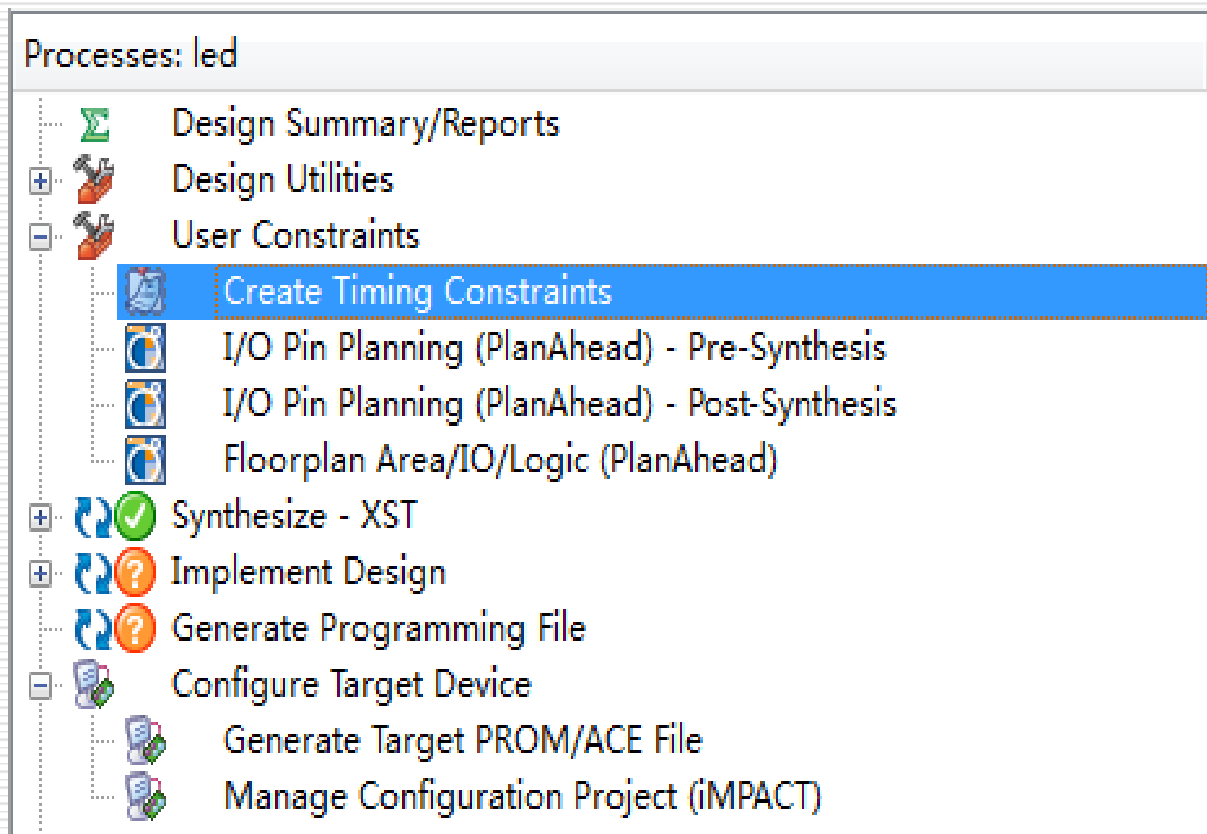
两输入与门——创建约束

约束主要包括全局时序约束（周期约束）和管脚约束，可以使用**图形化界面设置**和**直接编辑UCF文件**两种方式。一般初学者可以采用图形化界面设置的方式，比较直观，软件自动生成相关的约束而不需要手动编辑；如果是有一定经验的设计人员，对约束语法比较熟悉，且会涉及到一些高级约束时，则可以考虑使用直接编辑UCF文件的方式或者两种方法相结合。

两输入与门---创建约束

---图形化界面设置方式

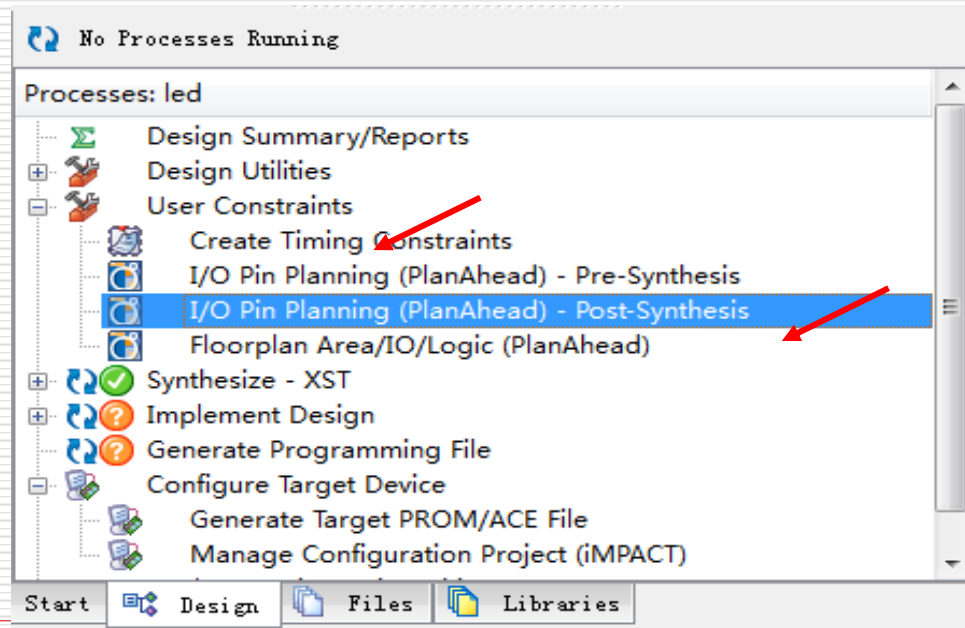
(1) 选中Implementation, 点击顶层文件
and2gate(and2gate.v), 按Yes, 打开Process窗口



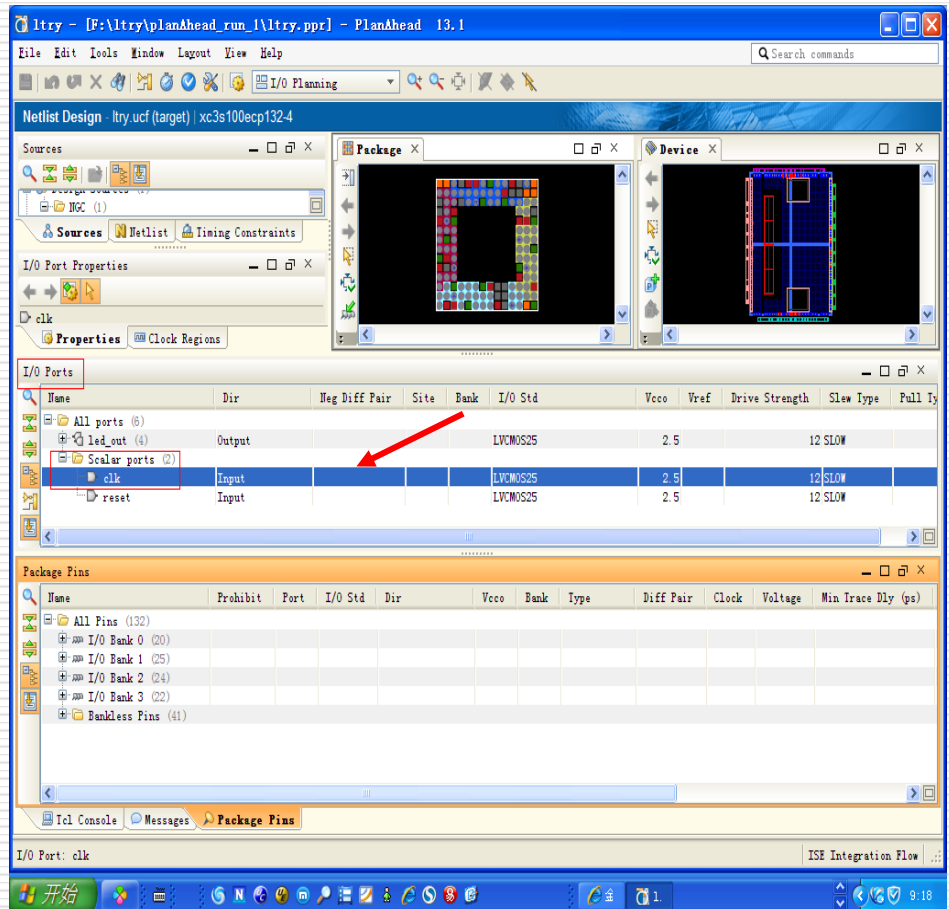
两输入与门---创建约束

---图形化界面设置方式

(2) 开始管脚约束：选择User Constraints中的I/O Pin Planning (PlanAhead) - Post-Synthesis，双击打开PlanAhead界面（本过程需要一定时间，视计算机配置要1分钟左右）。（如果没有.ucf文件，软件会提示是否需要自动创建ucf文件，点击Yes）



(3) 在打开的PlanAhead中对管脚进行设定.



两输入与门——创建约束

——图形化界面设置方式

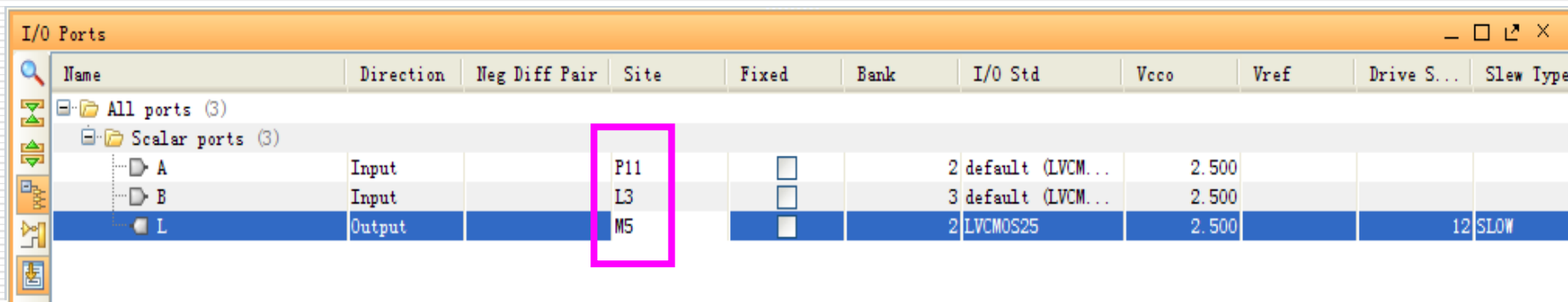
(4) 其他端口也是类似操作，最终的端口设定如下图所示。

选择A为拨码开关SW0；其管脚位置为P11；

选择B为拨码开关SW1；其管脚位置为L3；

选择L为发光二极管LD0；其管脚位置为M5；

注：如果更换了开发板，管脚约束会发生变化。这时代码可以不用修改，直接在PlanAhead图形化界面中对site做重新定义，具体参考实际板子的管脚分配说明。



Name	Direction	Neg Diff Pair	Site	Fixed	Bank	I/O Std	Vcco	Vref	Drive S...	Slew Type
All ports (3)										
Scalar ports (3)										
A	Input		P11	☐		2 default (LVCM...	2.500			
B	Input		L3	☐		3 default (LVCM...	2.500			
L	Output		M5	☐		2 LVCMOS25	2.500		12	SLOW

两输入与门——创建约束

——图形化界面设置方式

(5) 保存并退出PlanAhead。

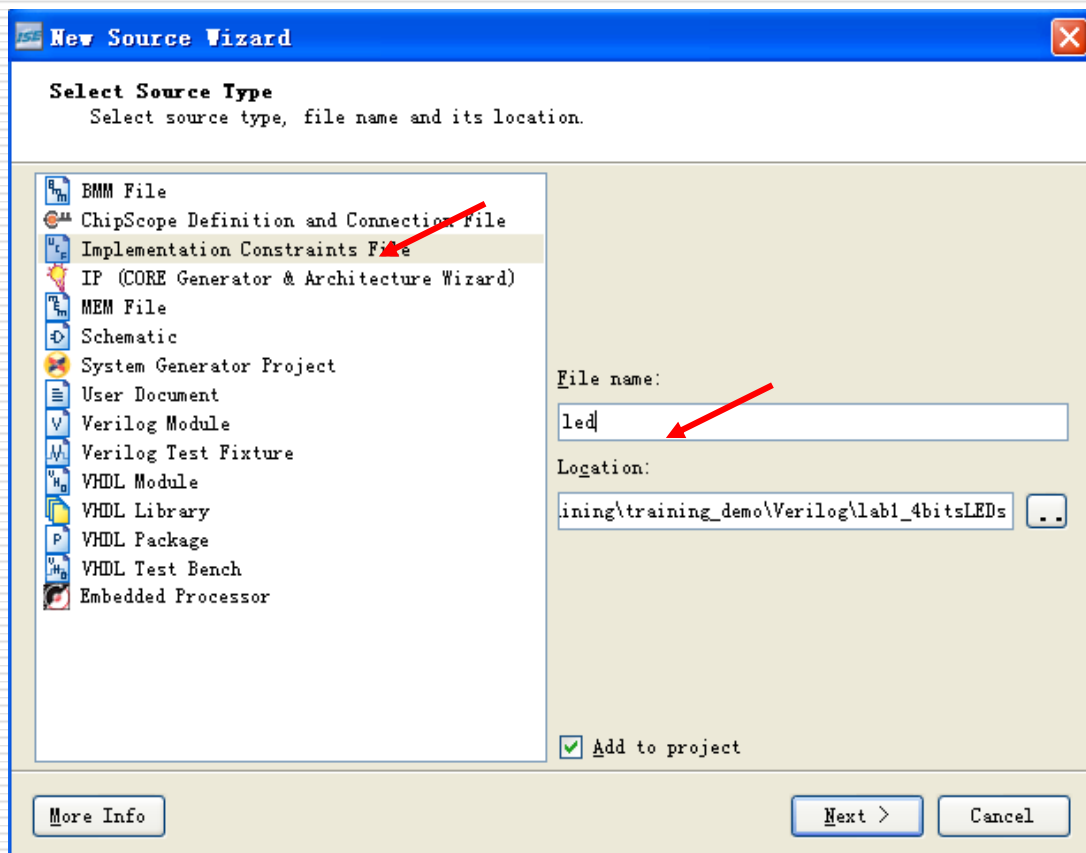
(6) 回到ISE界面，点选Sources窗口内的led.ucf, 点击process窗口里的user constraints, 双击edit constraints (txt)，可以看到之前所做的约束都自动保存在UCF文件中。

```
NET "L" IOSTANDARD = LVCMOS25;  
  
# PlanAhead Generated physical constraints  
  
NET "A" LOC = P11;  
NET "B" LOC = L3;  
NET "L" LOC = M5;
```

两输入与门---创建约束

---直接编辑UCF文件方式

(1) 选择Project ->New Source. 设定输入的文件格式为Implementation Constraints File, 文件名称为and2gate (自动储存为and2gate.UCF)



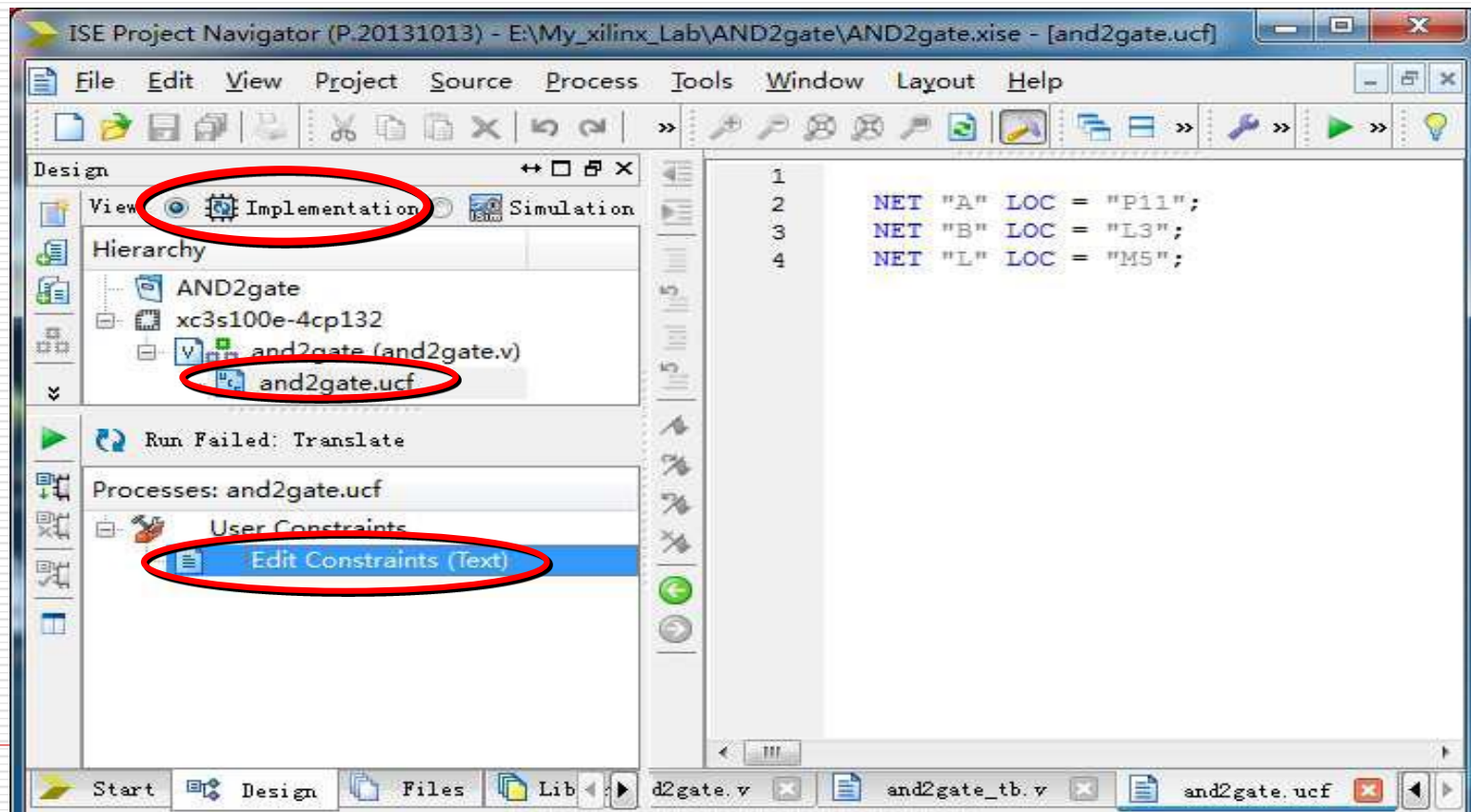
(2) 一直点击next
，最后点击finish

(3) 选择source
for Implementation
选项

两输入与门——创建约束

——直接编辑UCF文件方式

(2) 点选Sources窗口内的and2gate.ucf, 点击processes窗口里的user constraints, 双击edit constraints (txt), 直接在编辑窗口输入所需的约束后保存文件。



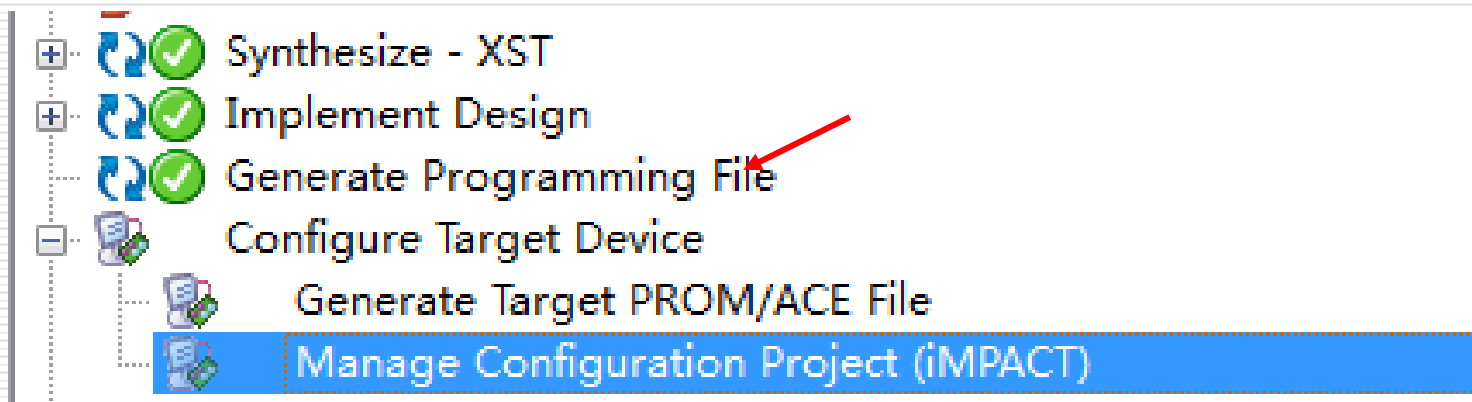
两输入与门——执行设计并生成下载的bit

约束创建完成之后，可执行设计并生成下载的bit文件。

执行设计并生成下载的bit文件，可直接双击Generate Programming File, 确认设计无误（即确认所有步骤后均出现绿色的打勾,表示无错或至少没有步骤出现红色打叉[X]的符号,表示没有不可容忍的错误）。

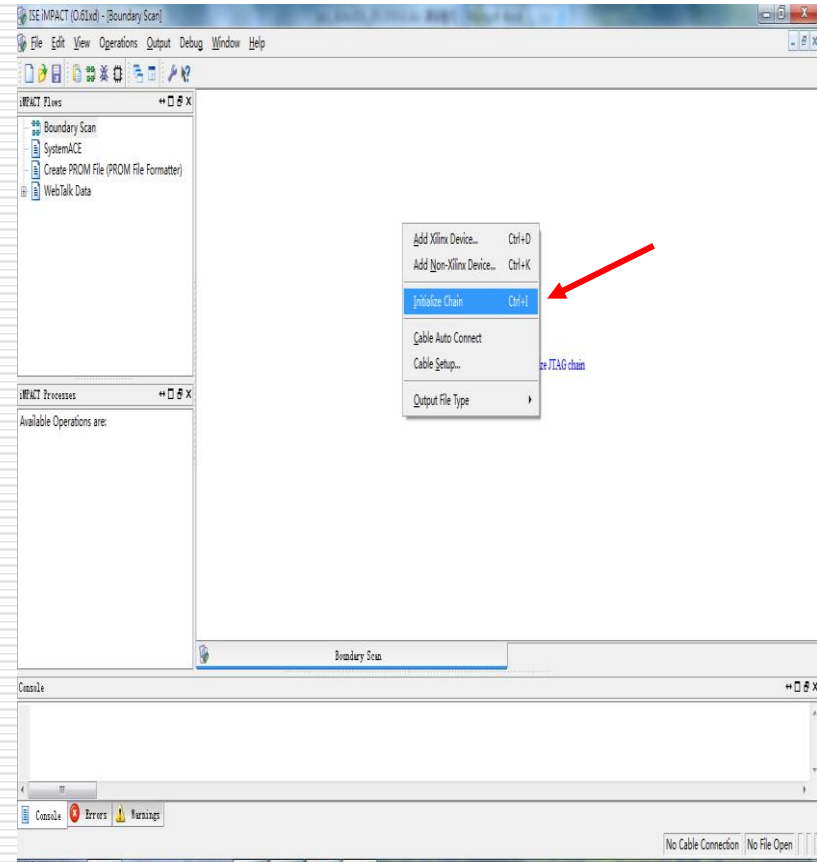
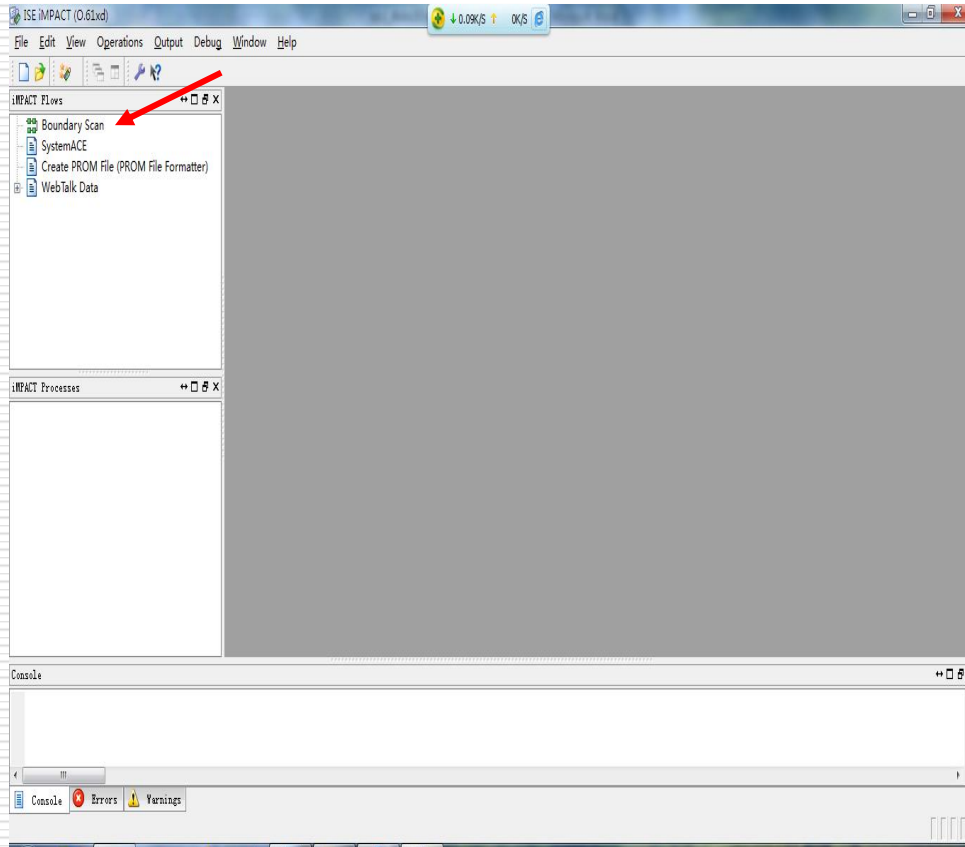
两输入与门——下载设计到实验板

(1) 确认执行设计无误并成功生成bit文件后, 就可以将and2gate.bit 写入到FPGA 内部, 此时连接板子的USB下载线, 并打开电源。点开“Configure Target Device → Manage Configuration Project (Impact)”



两输入与门——下载设计到实验板

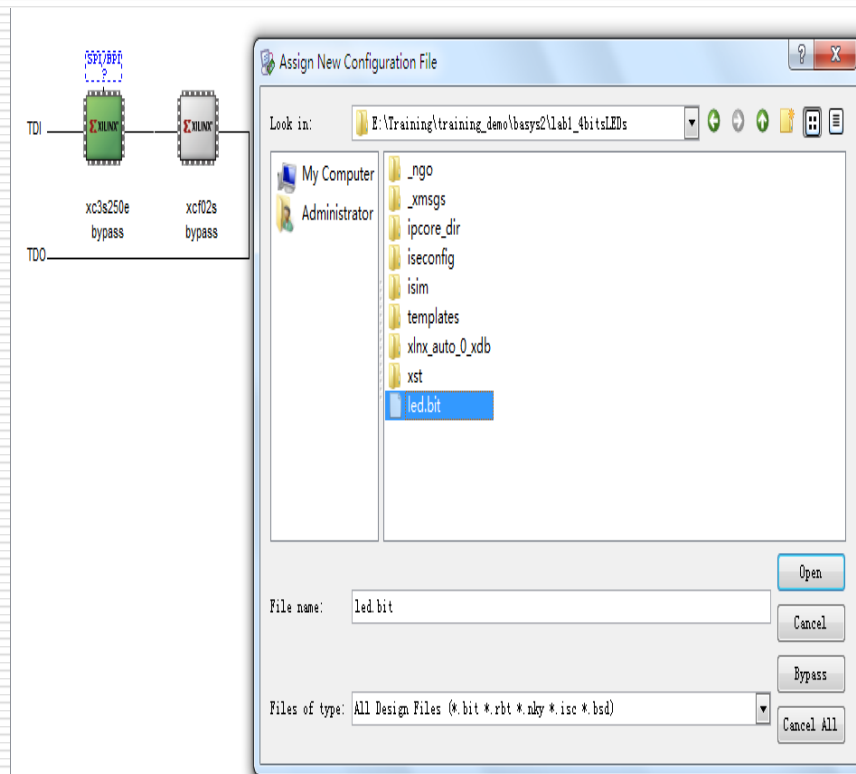
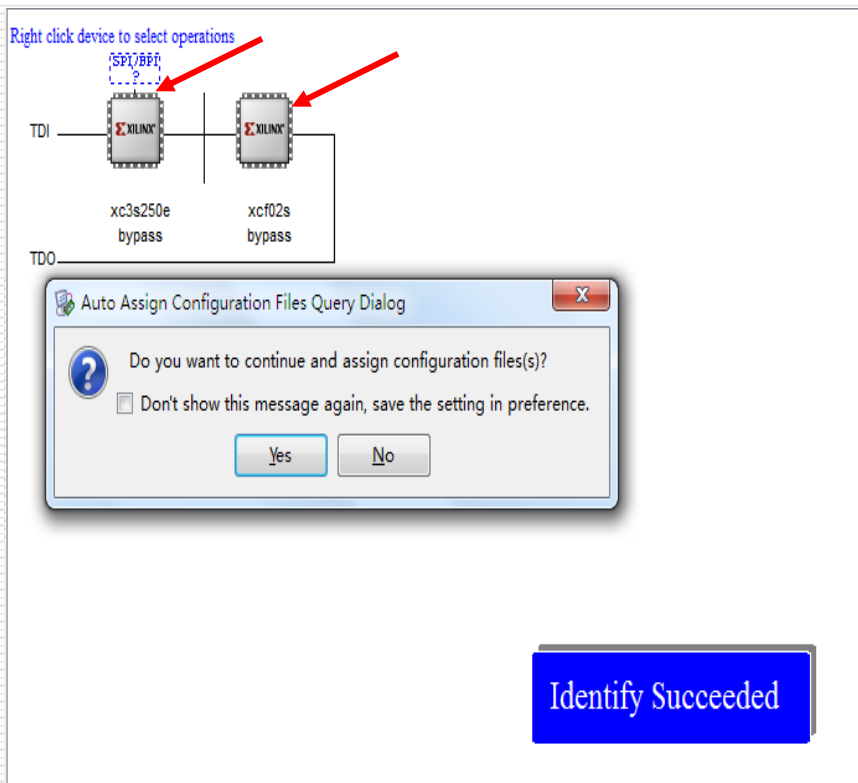
(2) 双击打开iMPACT软件



(3) 双击“Boundary Scan”，右边变成空白，此时点击右键选择“Initialize Chain”

两输入与门——下载设计到实验板

(4) 此时扫描到两个器件，一个是FPGA器件xc3s250e，一个是prom器件 xcf02s。

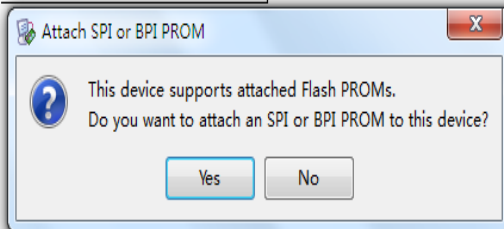
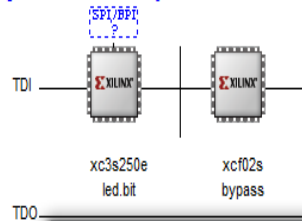


(5) 点击 “Yes”, 配置文件，寻找到工程的 and2gate.bit 文件

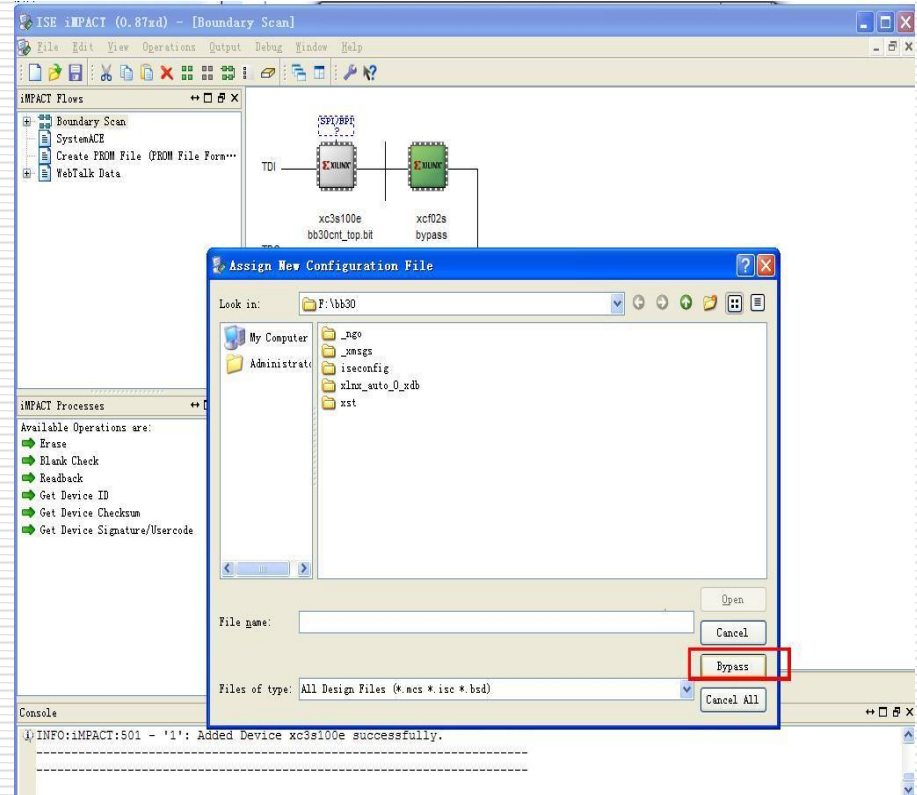
两输入与门——下载设计到实验板

(6) 点击“Open”，弹出是否需要配置SPI或BPI PROM，不需要点击No

Right click device to select operations



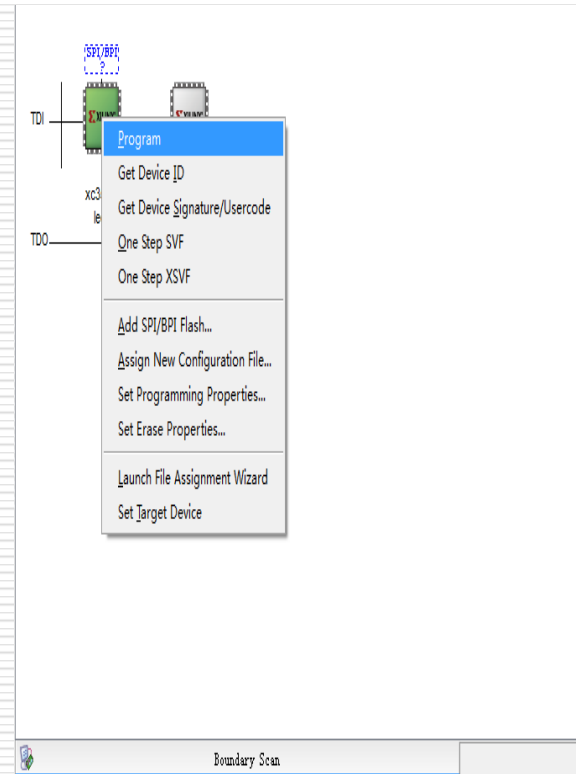
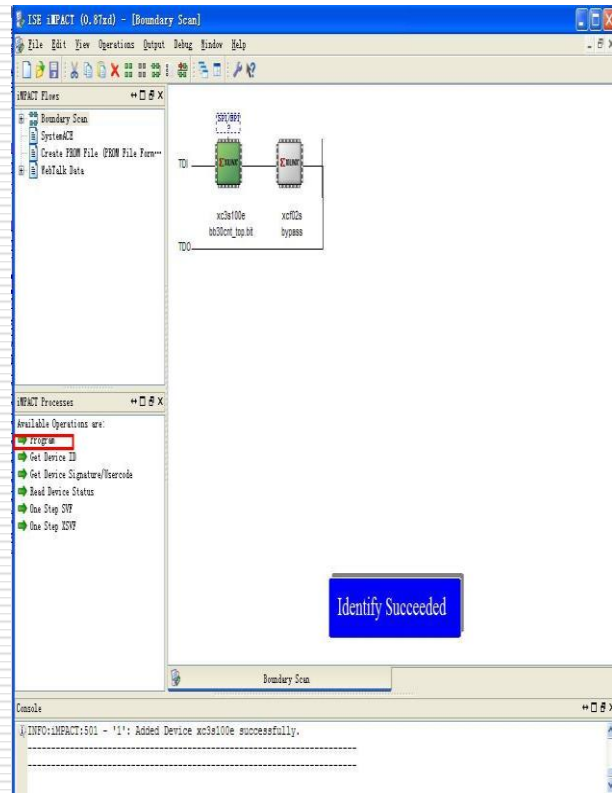
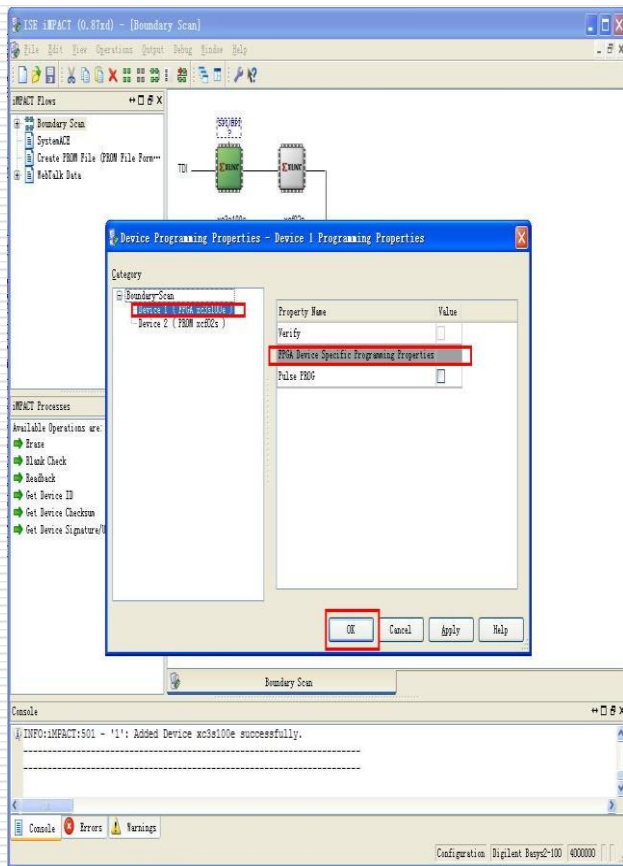
Identify Succeeded



(7) 接着在弹出窗口点击“Bypass”，并点击OK,然后选择fpga器件，点击右键选择“Program”

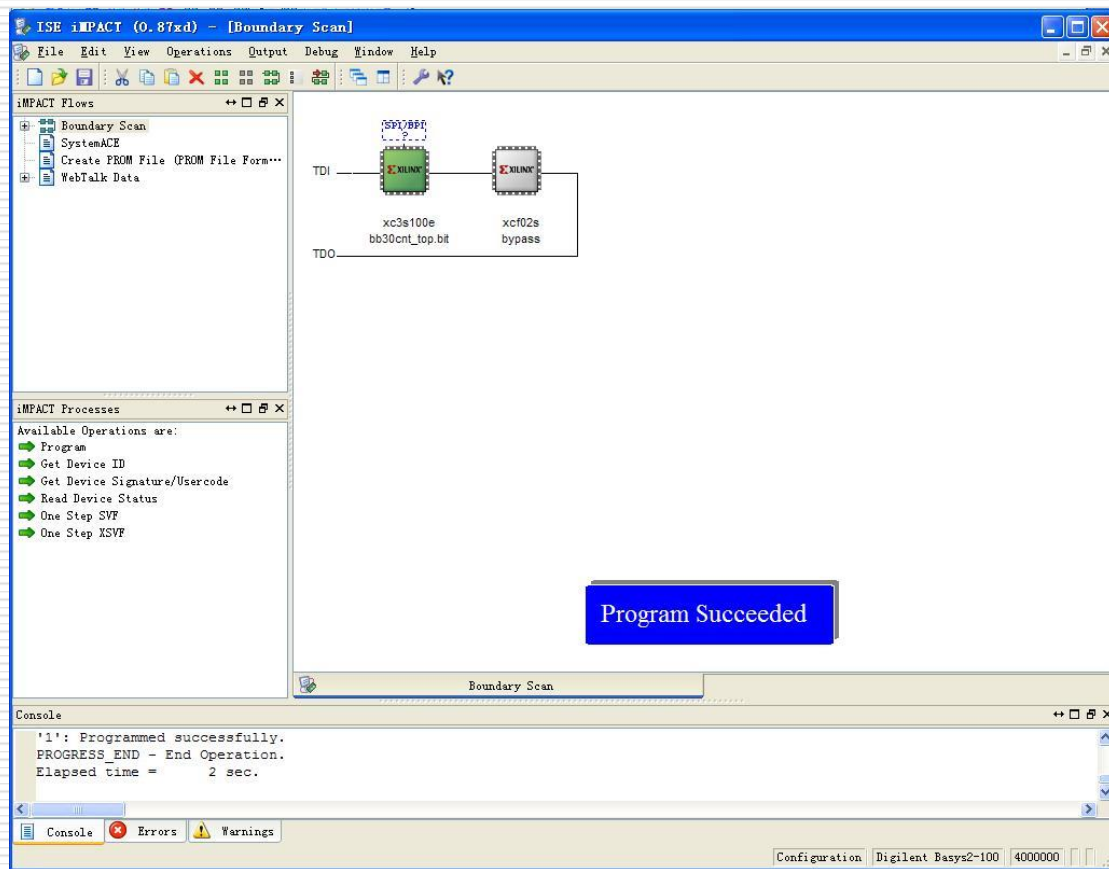
两输入与门——下载设计到实验板

(7) 接着在弹出窗口点击“Bypass”，并点击OK,然后选择fpga器件，点击右键选择“Program”



两输入与门——下载设计到实验板

(8) 最终会有“Program Succeeded”提示。查看开发板，观察发光管闪烁结果。



两输入与门——下载设计到实验板

也可利用Digilent公司提供Adept2.0进行下载

在完成设计验证之后,就可以将led.bit 写入到FPGA 内部,此时连接BASYS2实验板的USB 下载线到PC机, 并打开BASYS2实验板上的电源开关, 在PC机桌面点击“开始->Digilent->Adept ->adept”, 打开专门的下载工具。
